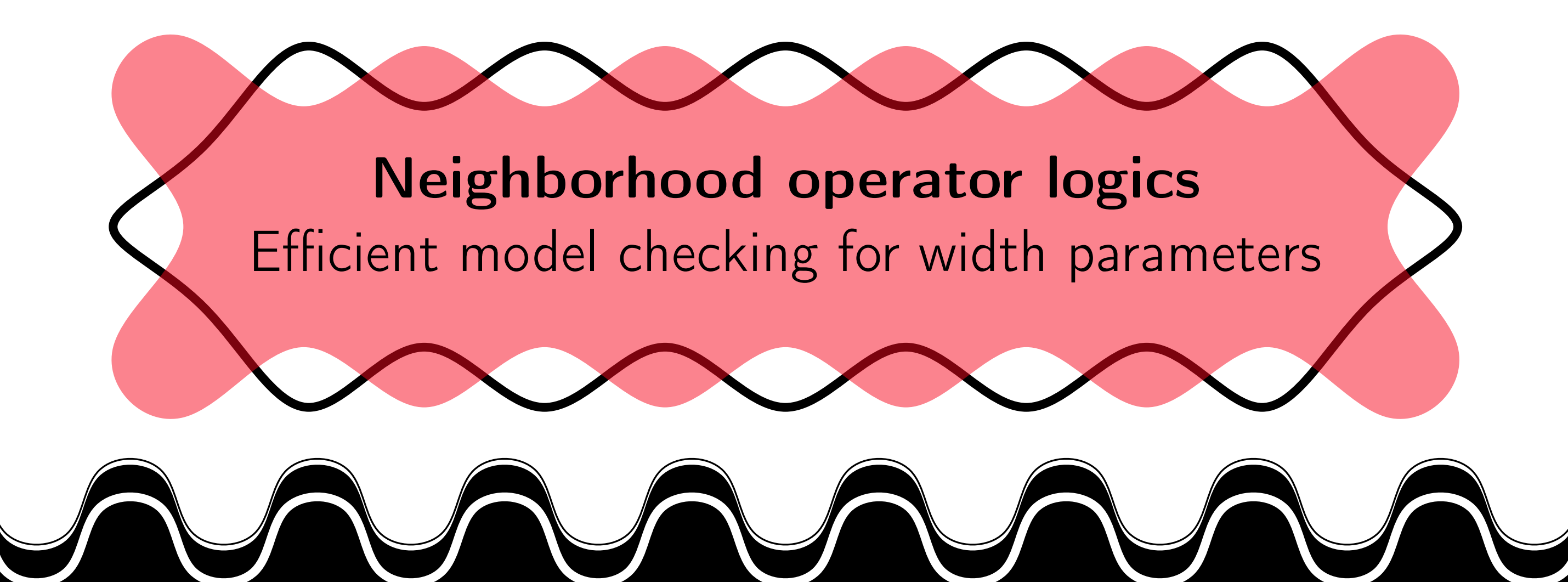


The slide features a decorative header with a black wavy line at the top. Below it is a light pink, cloud-like shape with a black outline. Inside this shape, the title and subtitle are written in black text. At the bottom of the slide, there is a black wavy line with a white outline.

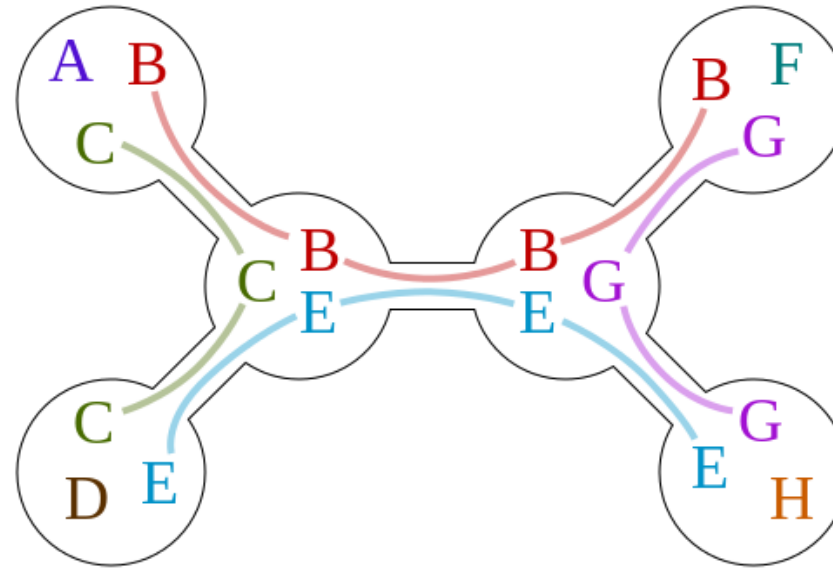
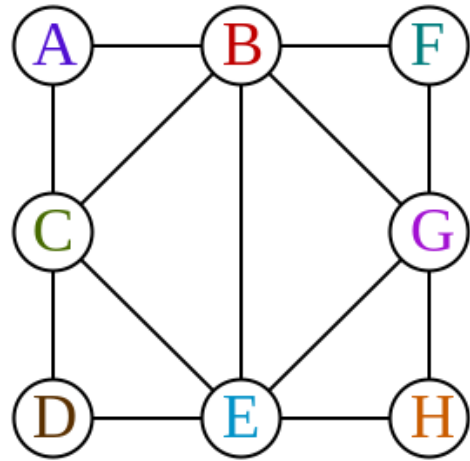
Neighborhood operator logics

Efficient model checking for width parameters

Benjamin Bergougnoux

Workshop on Modern Trends in Parameterized Complexity
Venice, April 22, 2026

Width parameters

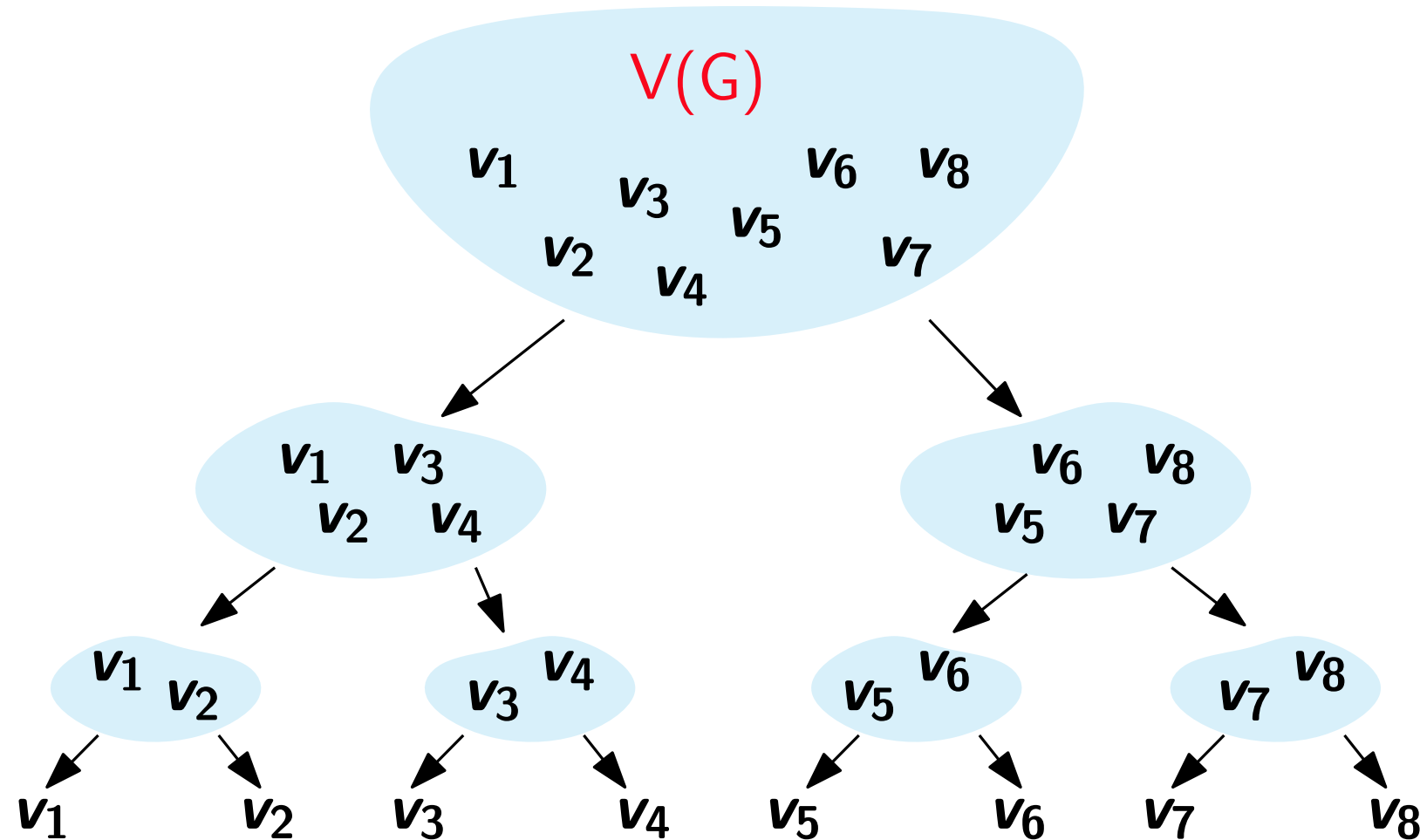


- **Width parameters:** treewidth, clique-width, rank-width, twin-width, mim-width,...
- ➔ **Divide and Conquer** on **combinatorial objects**
- ➔ Capture many **classical graph classes** (Interval graphs, Planar graphs)
- ➔ Generalize and unify many **tractability results** (Courcelle's theorem)

A small parenthesis: $\mathcal{F}$ -branchwidth

A great framework introduced by [\[Eiben, Ganian, Hamm, Jaffke, Kwon, ITCS 2022\]](#)

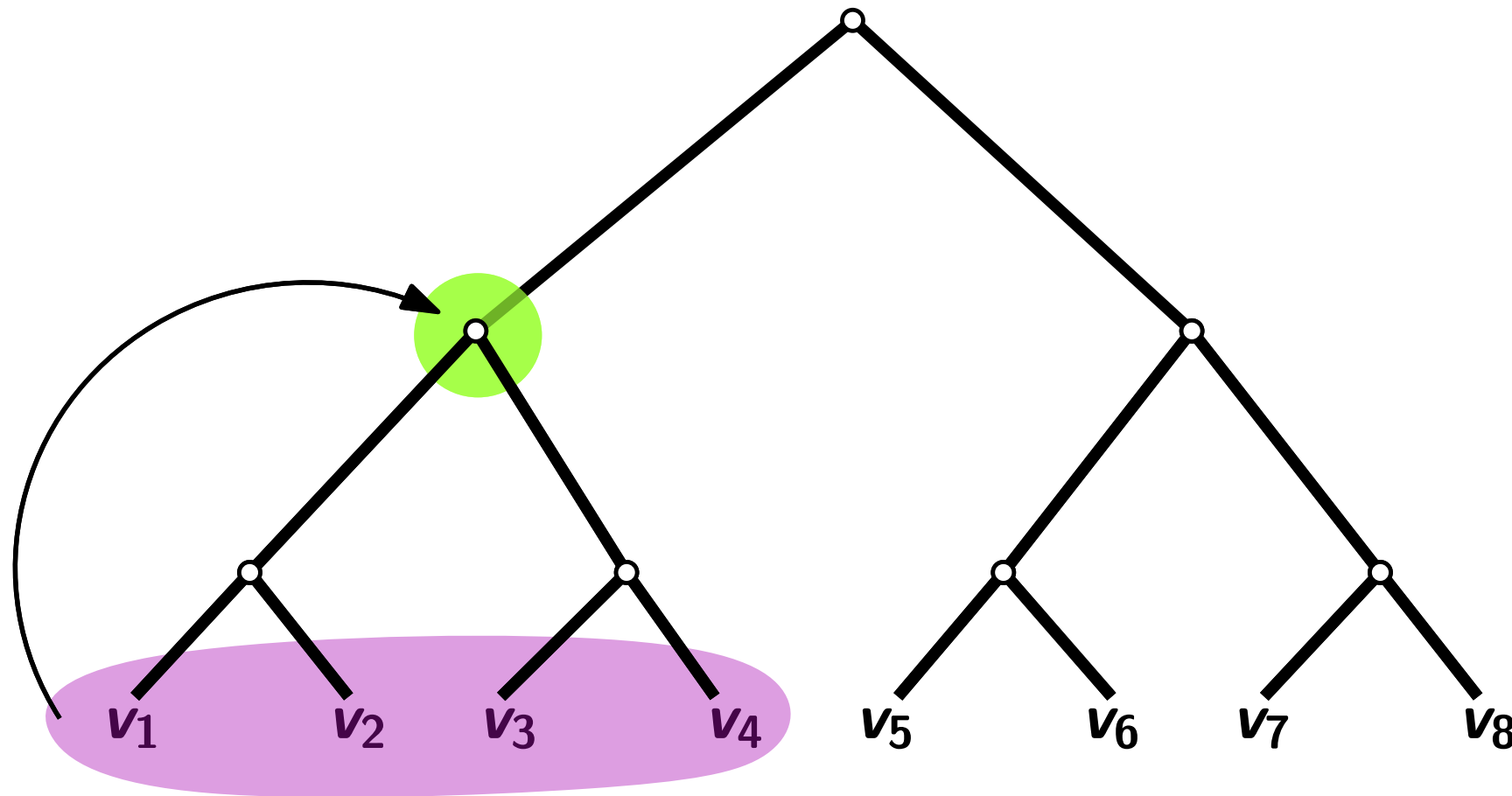
Layout of a graph: recursively cut **the vertex set** in two



A small parenthesis: $\mathcal{F}$ -branchwidth

A great framework introduced by [\[Eiben, Ganian, Hamm, Jaffke, Kwon, ITCS 2022\]](#)

Nodes of the layout $\longrightarrow$ vertex sets of G



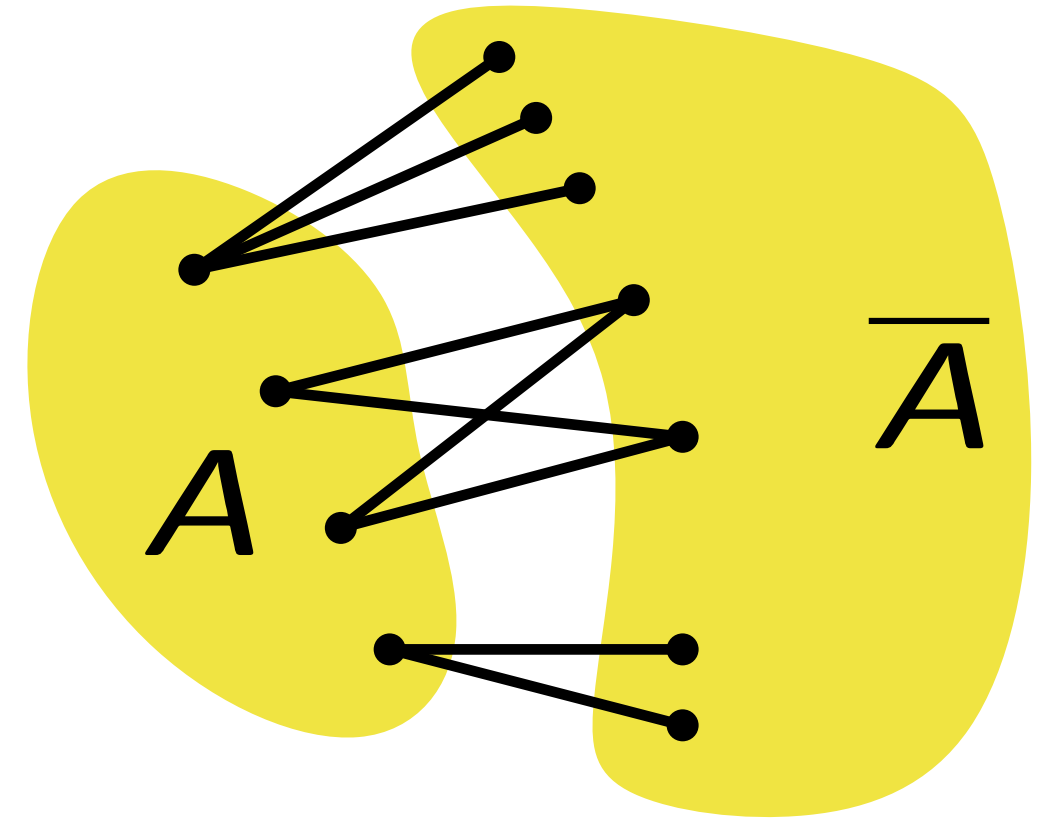
$\mathcal{F}$ -branchwidth of cuts

We measure the complexity of a cut with an infinite family $\mathcal{F}$ of bipartite graphs

We require $\mathcal{F}$ to be **partner-hereditary**

- Every vertex is paired with one on the other side
- $\mathcal{F}$ is close under taking subgraphs induced by subset of pairs

Example: $\mathcal{F}$ = class of matchings

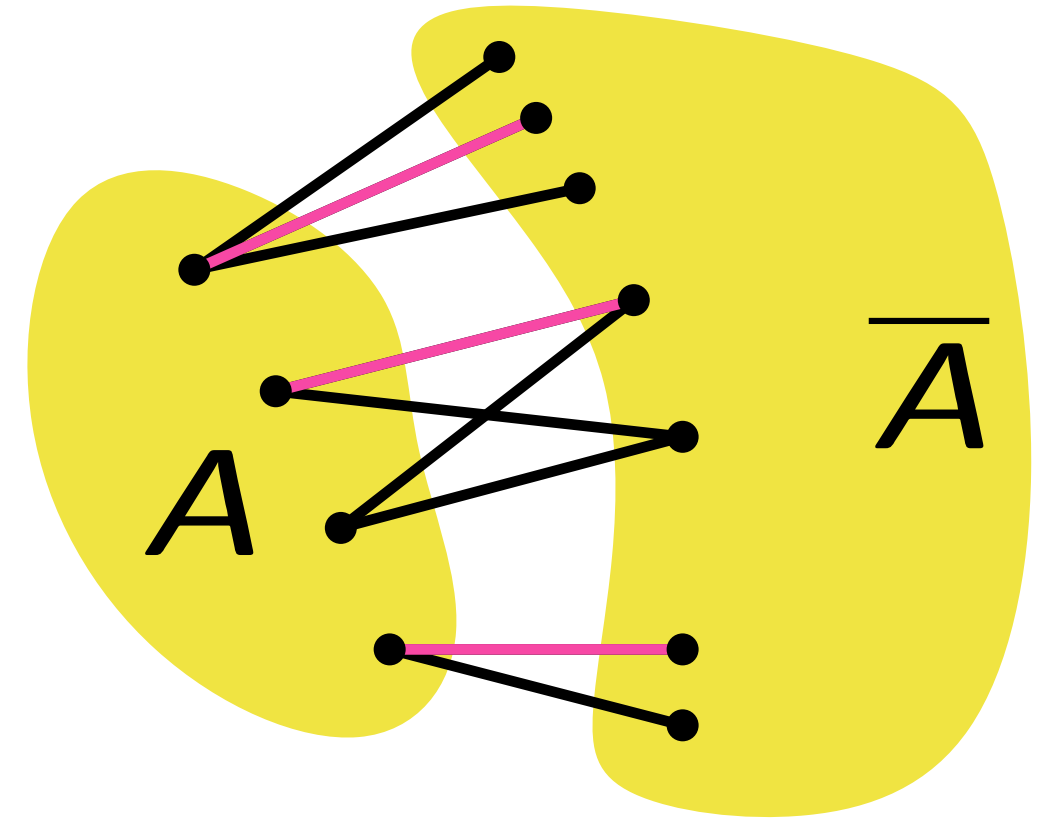


$\mathcal{F}$ -branchwidth of cuts

We measure the complexity of a cut with an infinite family $\mathcal{F}$ of bipartite graphs

We require $\mathcal{F}$ to be **partner-hereditary**

- Every vertex is paired with one on the other side
- $\mathcal{F}$ is close under taking subgraphs induced by subset of pairs



Example: $\mathcal{F}$ = class of matchings

$\mathcal{F}$ -bw(A) = largest k such that a $2k$ -vertex graph in $\mathcal{F}$ appears in $G[A, \bar{A}]$

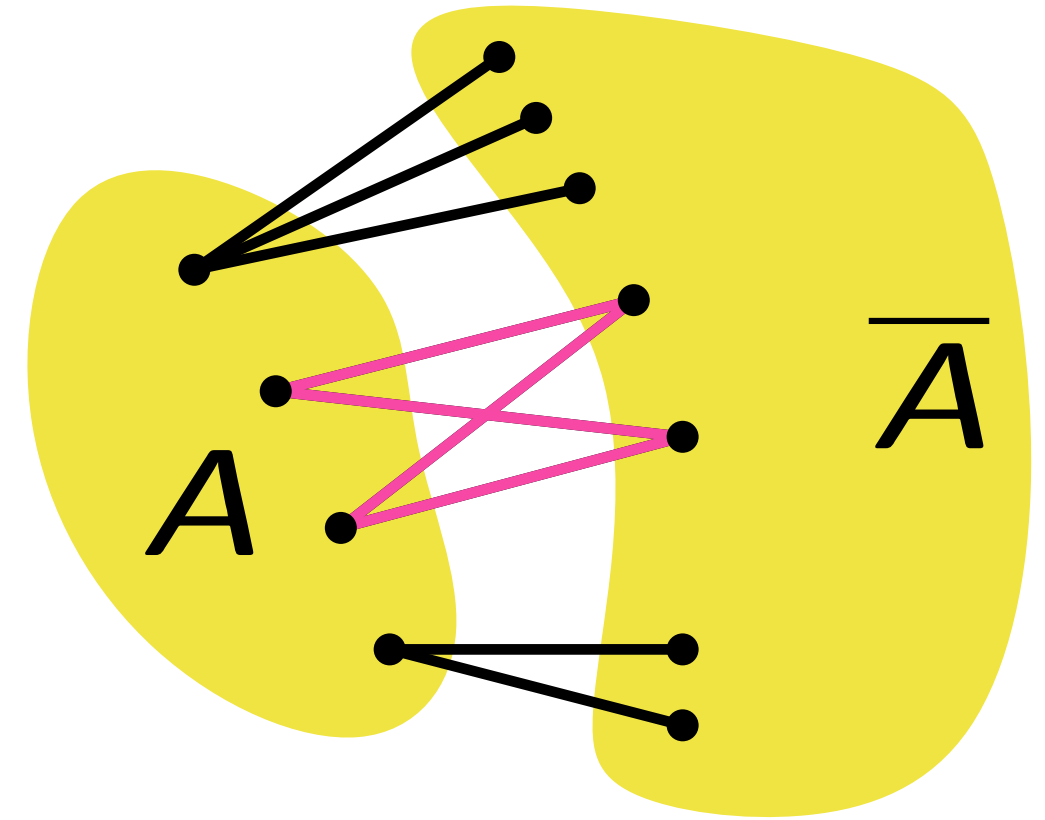
A.k.a. the **mim-width** of a cut

$\mathcal{F}$ -branchwidth of cuts

We measure the complexity of a cut with an infinite family $\mathcal{F}$ of bipartite graphs

We require $\mathcal{F}$ to be **partner-hereditary**

- Every vertex is paired with one on the other side
- $\mathcal{F}$ is close under taking subgraphs induced by subset of pairs



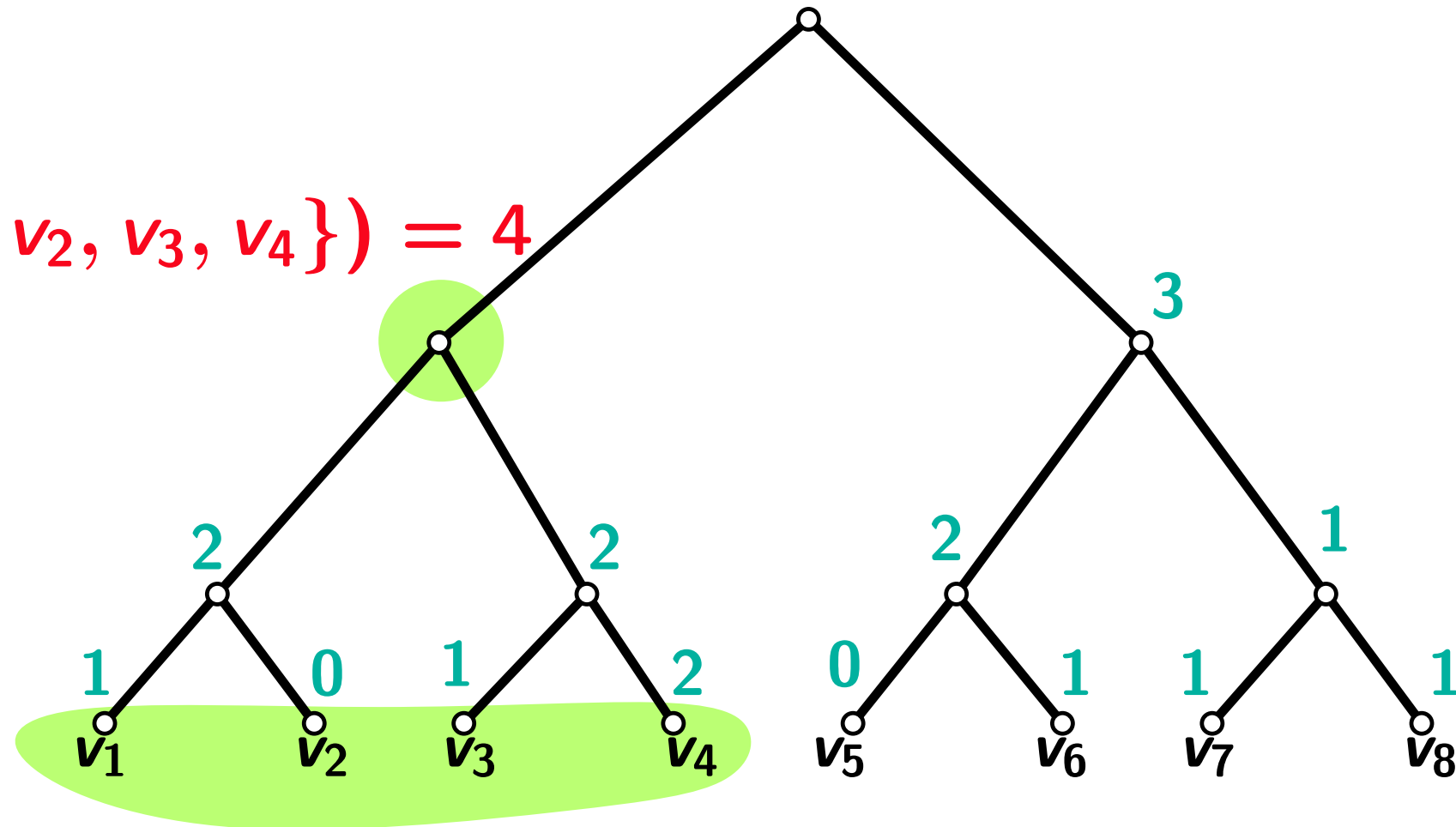
Example: $\mathcal{F}$ = class of bicliques

$\mathcal{F}\text{-bw}(A) = \text{largest } k \text{ such that a } 2k\text{-vertex graph in } \mathcal{F} \text{ appears in } G[A, \overline{A}]$

$\mathcal{F}$ -branchwidth

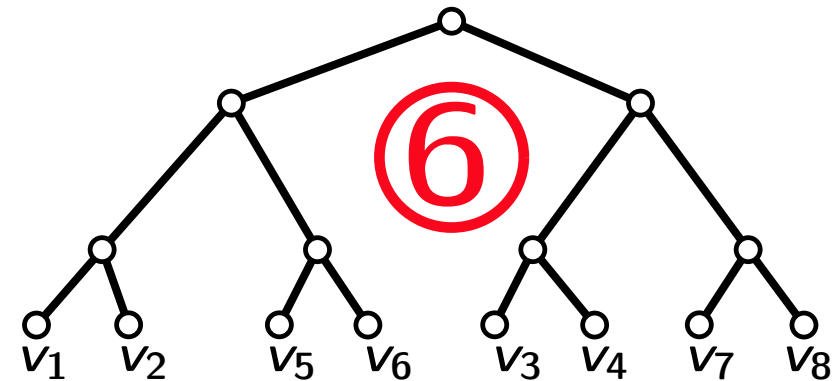
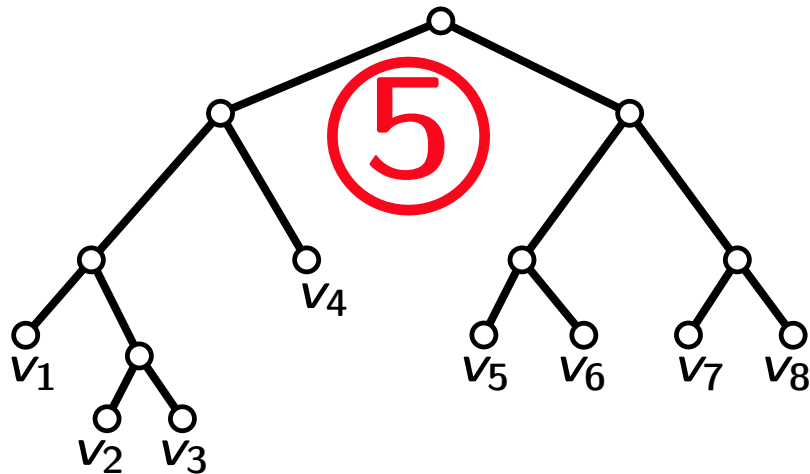
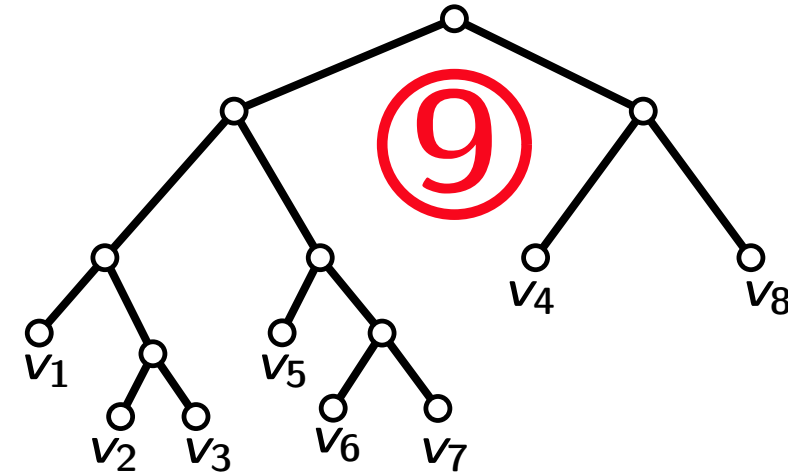
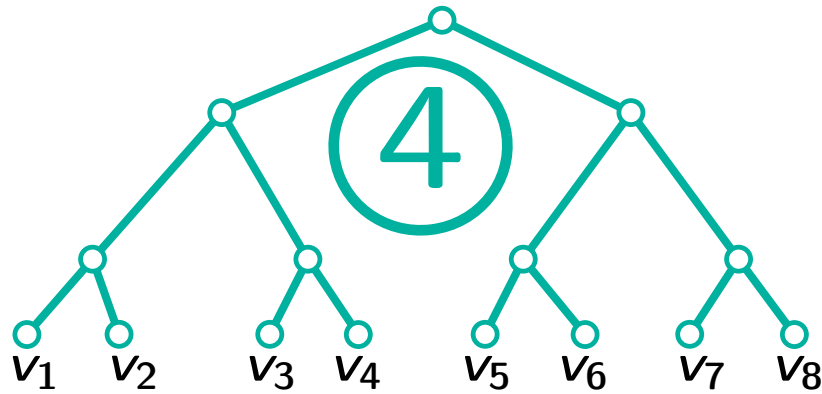
Width of a layout $D = \max f(\text{cuts})$

$$f(\{v_1, v_2, v_3, v_4\}) = 4$$



$\mathcal{F}$ -branchwidth

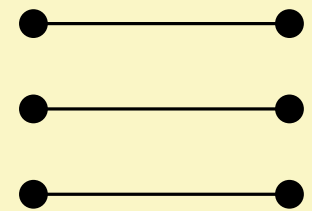
Width of a **graph** = **min** width of its **layouts**



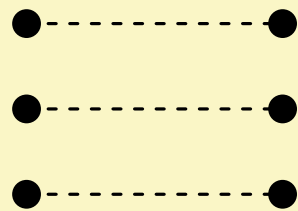
Modeling power of $\mathcal{F}$ -branchwidth

[Eiben, Ganian, Hamm, Jaffke, Kwon, ITCS 2022]

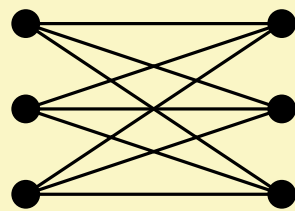
Any $\mathcal{F}$ -branchwidth is equivalent to a $\mathcal{F}'$ -branchwidth where $\mathcal{F}'$ is a combination of



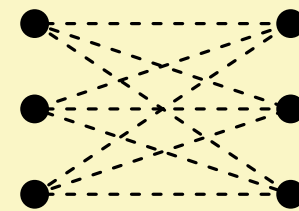
matchings



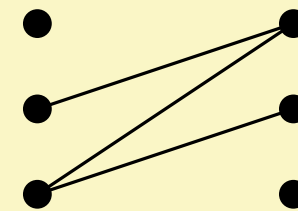
anti-matchings



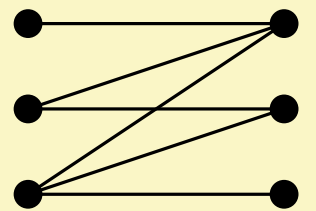
bicliques



edgeless graphs



skewed chains

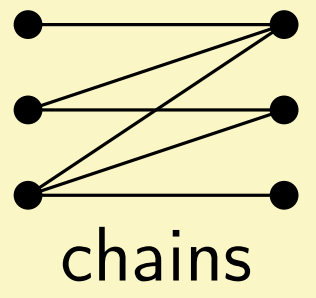
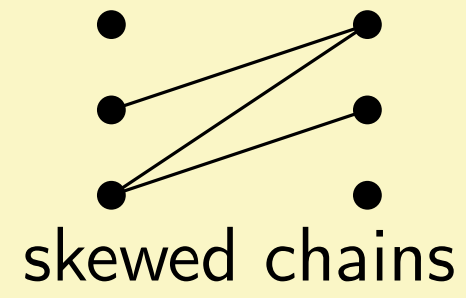
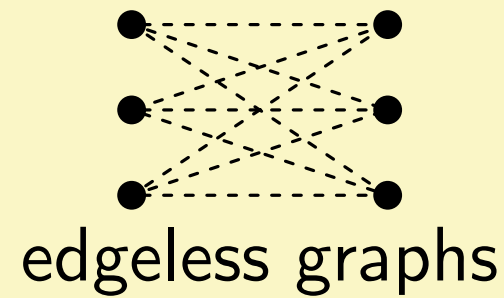
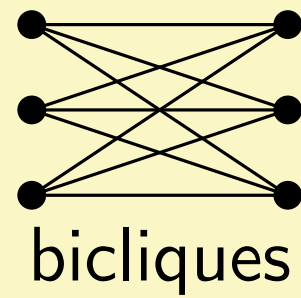
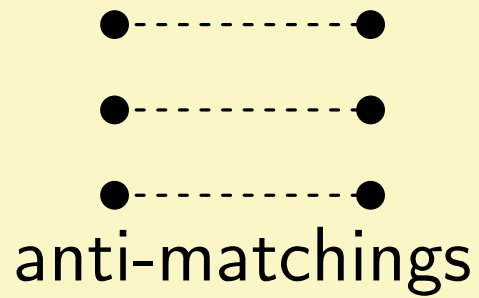
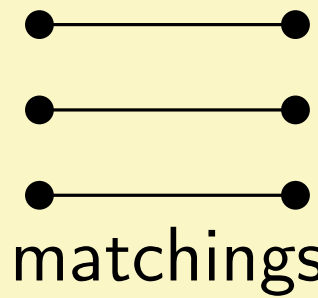


chains

Modeling power of $\mathcal{F}$ -branchwidth

[Eiben, Ganian, Hamm, Jaffke, Kwon, ITCS 2022]

Any $\mathcal{F}$ -branchwidth is equivalent to a $\mathcal{F}'$ -branchwidth where $\mathcal{F}'$ is a combination of

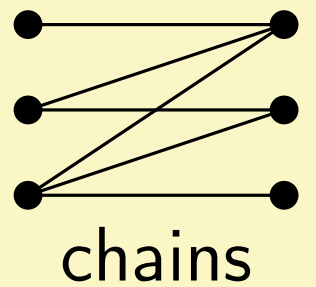
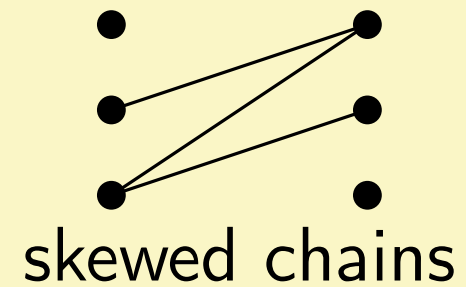
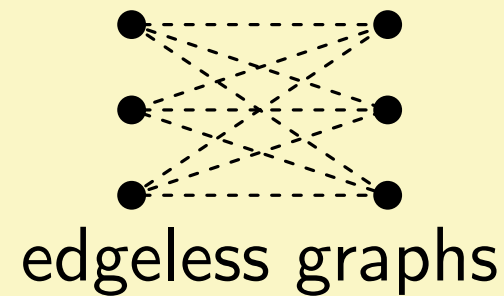
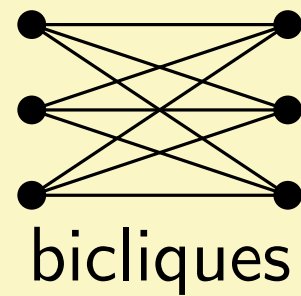
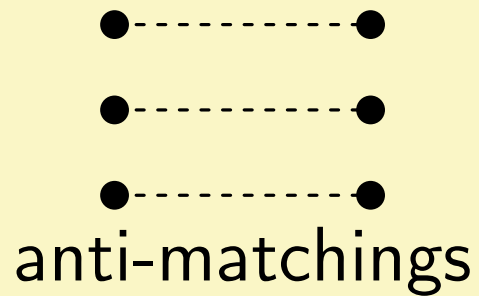
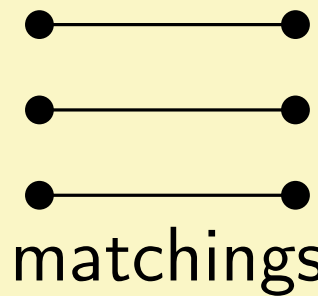


- There at most $2^6 - 1$ subfamilies of $\mathcal{F}$ -branchwidths to consider

Modeling power of $\mathcal{F}$ -branchwidth

[Eiben, Ganian, Hamm, Jaffke, Kwon, ITCS 2022]

Any $\mathcal{F}$ -branchwidth is equivalent to a $\mathcal{F}'$ -branchwidth where $\mathcal{F}'$ is a combination of

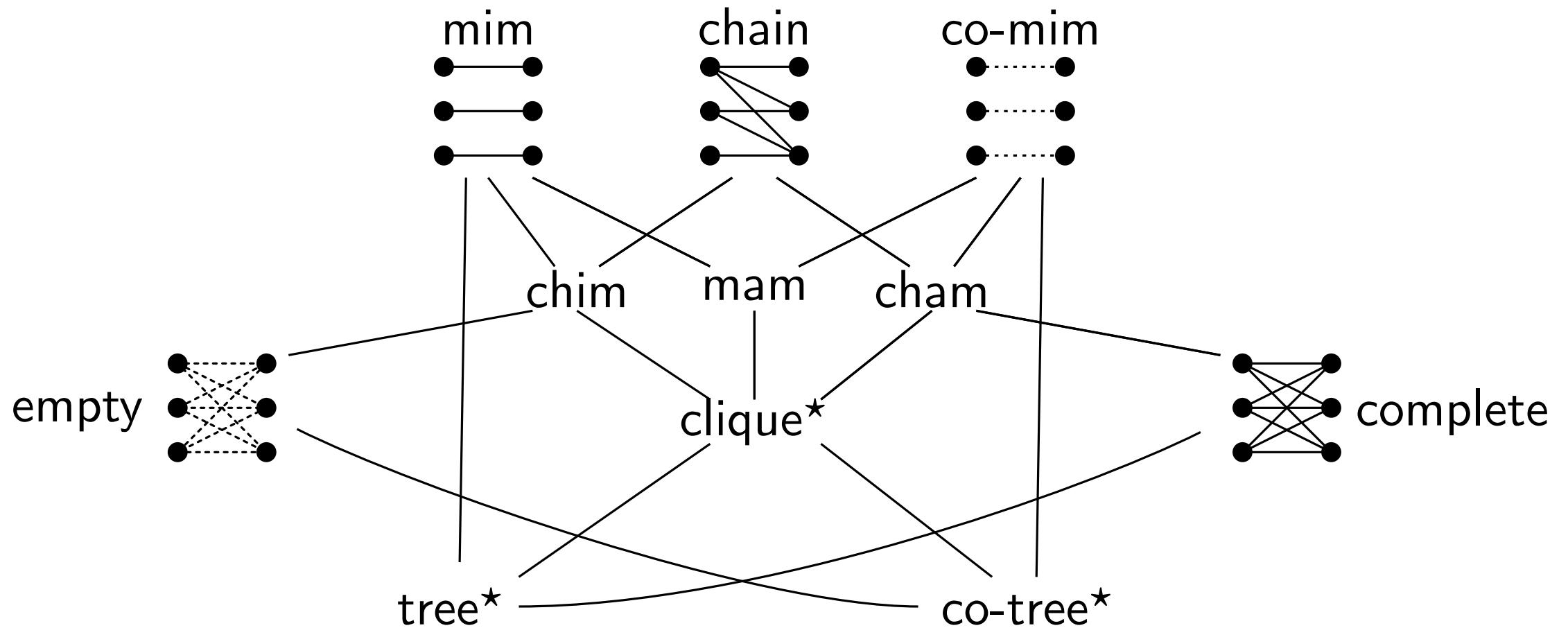


- There at most $2^6 - 1$ subfamilies of $\mathcal{F}$ -branchwidths to consider
- It includes widths equivalent to **treewidth**, **clique-width**, **mim-width**

The Eleven

[B., Hamm, Jaffke, Lima, ESA 2024]

There is a family of **11 pairwise non-equivalent** $\mathcal{F}$ -branchwidths such that any $\mathcal{F}$ -branchwidth is equivalent to one of the 11

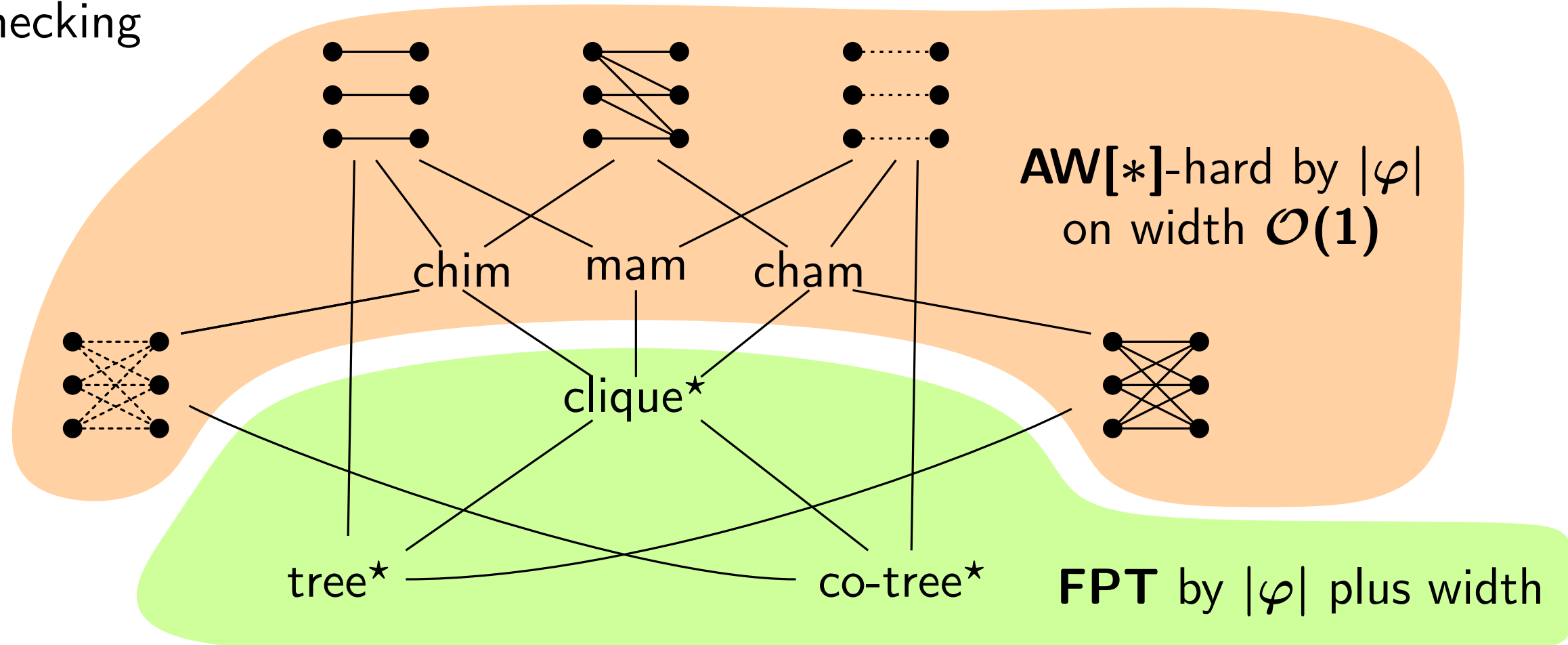


The Eleven

[B., Hamm, Jaffke, Lima, ESA 2024]

There is a family of **11 pairwise non-equivalent** $\mathcal{F}$ -branchwidths such that any $\mathcal{F}$ -branchwidth is equivalent to one of the 11

FO-model checking

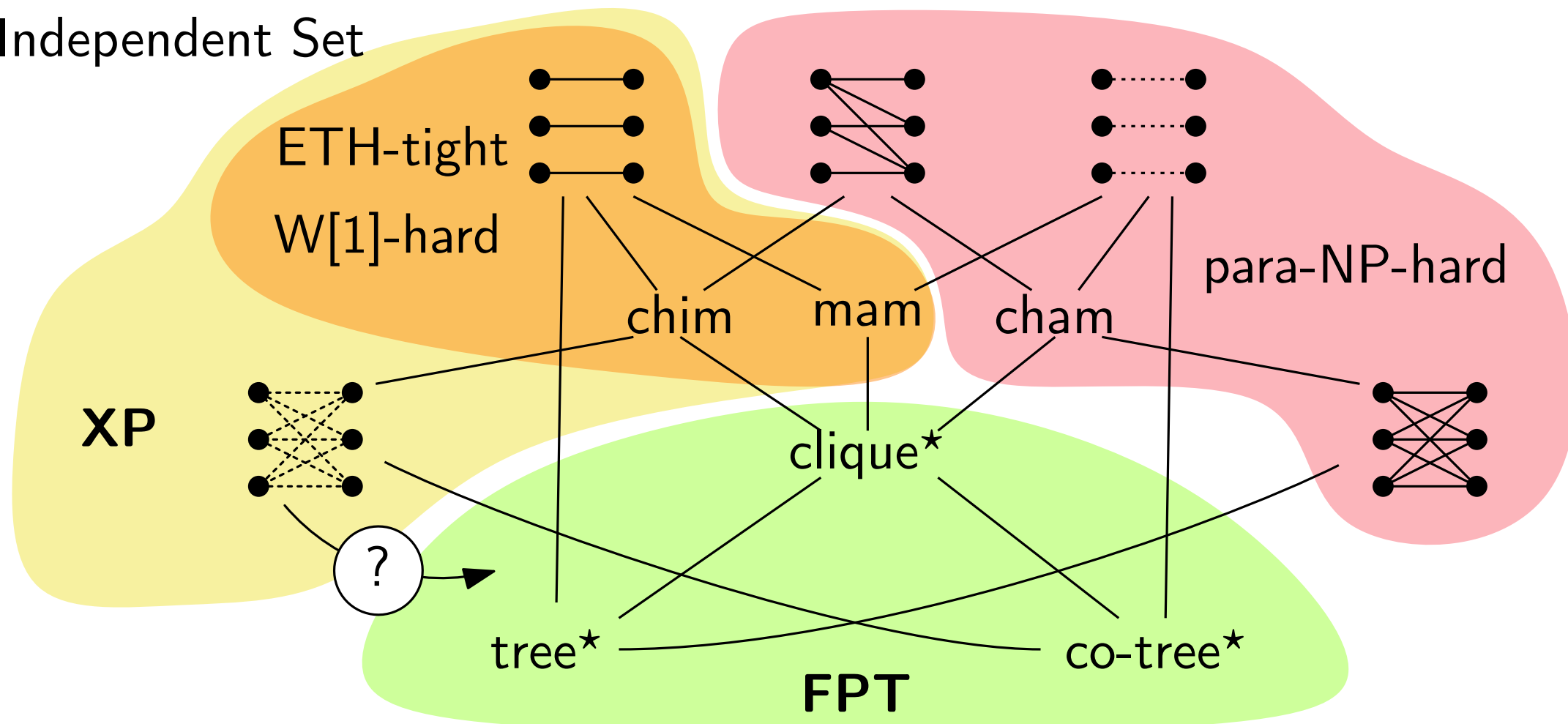


The Eleven

[B., Hamm, Jaffke, Lima, ESA 2024]

There is a family of **11 pairwise non-equivalent** $\mathcal{F}$ -branchwidths such that any $\mathcal{F}$ -branchwidth is equivalent to one of the 11

Max. Independent Set



Meta-algorithmic theorems

[Courcelle, 1990]

Every **problem expressible in MSO_2** is solvable in time $f(\text{tw}, |\varphi|)n$ given a tree decomposition of width tw

Meta-algorithmic theorems

[Courcelle, 1990]

Every **problem expressible in MSO_2** is solvable in time $f(\text{tw}, |\varphi|)n$ given a tree decomposition of width tw

$$\text{Problem: } f(k, |\varphi|) = 2^{2^{\cdot^{2^k}}}$$

Probably **unavoidable** even for model-checking **FO** on trees! [Flick et Grohe, 2004]

Meta-algorithmic theorems

[Courcelle, 1990]

Every **problem expressible in MSO_2** is solvable in time $f(\text{tw}, |\varphi|)n$ given a tree decomposition of width tw

$$\text{Problem: } f(k, |\varphi|) = 2^{2^{\cdot 2^k}}$$

Probably **unavoidable** even for model-checking **FO** on trees! [Flick et Grohe, 2004]

For **many** problems, we can obtain **more efficient** algorithms!

Many **NP-hard problems** can be solved in time $2^{\mathcal{O}(\text{tw})} n^{\mathcal{O}(1)}$

→ Max. Independent Set, Min. Dominating Set, Hamiltonian Cycle, 3-coloring,...

Efficient meta-algorithmic theorems

Algorithmic meta-theorems like **Courcelle's theorem** does not explains why so **many problems** can be solved in time $2^{\mathcal{O}(\text{tw})} n^{\mathcal{O}(1)}$

Can we explain these efficient algorithms via an **algorithmic meta-theorem**?

Efficient meta-algorithmic theorems

Algorithmic meta-theorems like **Courcelle's theorem** does not explains why so **many problems** can be solved in time $2^{\mathcal{O}(\text{tw})} n^{\mathcal{O}(1)}$

Can we explain these efficient algorithms via an **algorithmic meta-theorem**?

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+C** are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

ETH
tight

- ECML = *Existential Counting Modal Logic with connectivity predicates*

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

$$\exists x \subseteq v (|X| \leq k) \wedge \forall_v \neg X \Rightarrow \Diamond^{\mathbb{N} \setminus \{0\}} X$$

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

$$\exists X \subseteq V (|X| \leq k) \wedge \forall v \neg X \Rightarrow \Diamond^{\mathbb{N} \setminus \{0\}} X$$

→ true iff the active vertex belongs to X

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

$$\exists X \subseteq V (|X| \leq k) \wedge \forall v \neg X \Rightarrow \Diamond^{N \setminus \{0\}} X$$

→ true iff the active vertex has a neighbors in X

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

$$\exists_{X \subseteq V} (|X| \leq k) \wedge \forall_v \neg X \Rightarrow \Diamond^{\mathbb{N} \setminus \{0\}} X$$

→ true iff the active vertex is in X or it has at least one neighbor in X

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

$$\exists X \subseteq V (|X| \leq k) \wedge \forall v \neg X \Rightarrow \Diamond^{\mathbb{N} \setminus \{0\}} X$$

→ true iff the following **formula** is true for every v as an active vertex

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

- $\Diamond^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
- $\Box^\sigma \varphi$ means $\neg \Diamond^\sigma \neg \varphi$

$$\exists X \subseteq V (|X| \leq k) \wedge \forall v \neg X \Rightarrow \Diamond^{\mathbb{N} \setminus \{0\}} X$$

→ true iff there exists a **k**-dominating set

Other widths parameters

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+C** with connectivity predicates are decidable in time $2^{\mathcal{O}(tw \cdot |\varphi|)} n^{\mathcal{O}(1)}$

What about other width parameters?

Other widths parameters

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+C** with connectivity predicates are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

What about other width parameters?

- Clique-width: many problems can be solved in time $2^{\mathcal{O}(\text{cw})} n^{\mathcal{O}(1)}$

Other widths parameters

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+C** with connectivity predicates are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

What about other width parameters?

- Clique-width: many problems can be solved in time $2^{\mathcal{O}(\text{cw})} n^{\mathcal{O}(1)}$
- Rank-width: many problems can be solved in time $2^{\mathcal{O}(\text{rw}^2)} n^{\mathcal{O}(1)}$

Other widths parameters

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+C** with connectivity predicates are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

What about other width parameters?

- Clique-width: many problems can be solved in time $2^{\mathcal{O}(\text{cw})} n^{\mathcal{O}(1)}$
- Rank-width: many problems can be solved in time $2^{\mathcal{O}(\text{rw}^2)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: many problems can be solved in time $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}})} n^{\mathcal{O}(1)}$

Other widths parameters

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+C** with connectivity predicates are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

What about other width parameters?

- Clique-width: many problems can be solved in time $2^{\mathcal{O}(\text{cw})} n^{\mathcal{O}(1)}$
- Rank-width: many problems can be solved in time $2^{\mathcal{O}(\text{rw}^2)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: many problems can be solved in time $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}})} n^{\mathcal{O}(1)}$
- Mim-width: many problems can be solved in time $n^{\mathcal{O}(\text{mim})}$

Solution: NEO logics

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$

Solution: NEO logics

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(cw \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(rw^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $rw_{\mathbb{Q}}^{\mathcal{O}(rw_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(mim \cdot |\varphi|)}$

- **NEO** = *Neighborhood operator*
- **FcoF** = finite and co-finite
- **+AC** = Acyclicity and connectivity constraints

Solution: NEO logics

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$

- **NEO** = *Neighborhood operator*
- **FcoF** = finite and co-finite
- **+AC** = Acyclicity and connectivity constraints

In fact: **NEO** logics are introduced in [B., Checkan, Stamoulis, SODA 2026]

[B., Dreier, Jaffke, SODA 2023] introduced **DN** $\equiv$ **NEO[FcoF]**+ distance neigh. op.

NEO logics: How to dodge the exponential tower?



NEO logics: starting point

Fully Existential MSO_2 :

- Variables for vertex/edge sets and vertices/edges

NEO logics: starting point

Fully Existential MSO_2 :

- Variables for vertex/edge sets and vertices/edges
- $\exists$, $\wedge$ and $\vee$

NEO logics: starting point

Fully Existential MSO_2 :

- Variables for vertex/edge sets and vertices/edges
- $\exists$, $\wedge$ and $\vee$
- Colors on vertices and edges

NEO logics: starting point

Fully Existential MSO_2 :

- Variables for vertex/edge sets and vertices/edges
- $\exists$, $\wedge$ and $\vee$
- Colors on vertices and edges
- **adjacent**(x, y) and $\in$

NEO logics: starting point

Fully Existential MSO_2 :

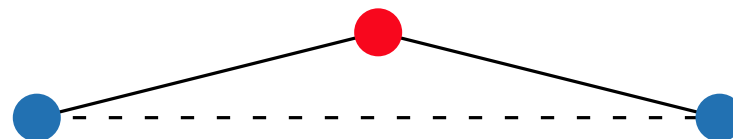
- Variables for vertex/edge sets and vertices/edges
- $\exists$, $\wedge$ and $\vee$
- Colors on vertices and edges
- **adjacent(x, y)** and $\in$
- **Negations ($\neg$) only on quantifier free formulas**

NEO logics: starting point

Fully Existential MSO_2 :

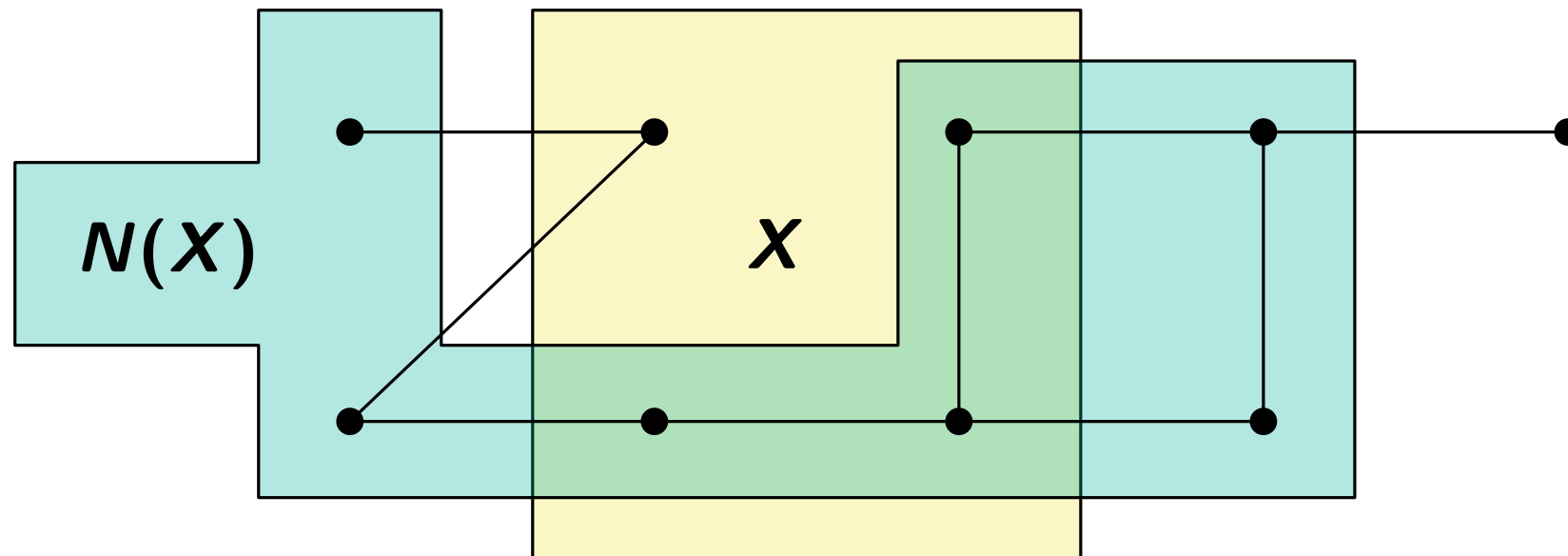
- Variables for vertex/edge sets and vertices/edges
- $\exists$, $\wedge$ and $\vee$
- Colors on vertices and edges
- $\text{adjacent}(x, y)$ and $\in$
- **Negations ($\neg$) only on quantifier free formulas**

$\exists x \in \text{B}, y \in \text{R}, z \in \text{B} \text{ adjacent}(x, y) \wedge \text{adjacent}(y, z) \wedge \neg \text{adjacent}(x, z)$



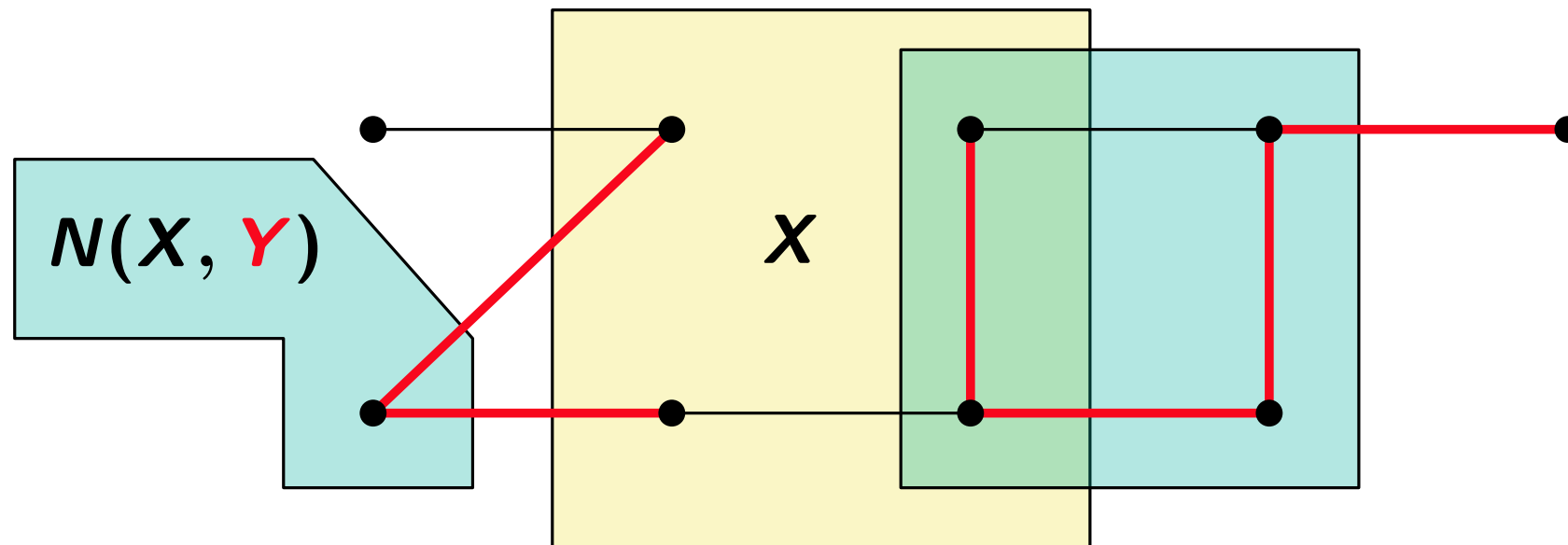
NEO logics: neighborhood operators

- $N(X)$ = all the vertices adjacent to at least one vertex of X



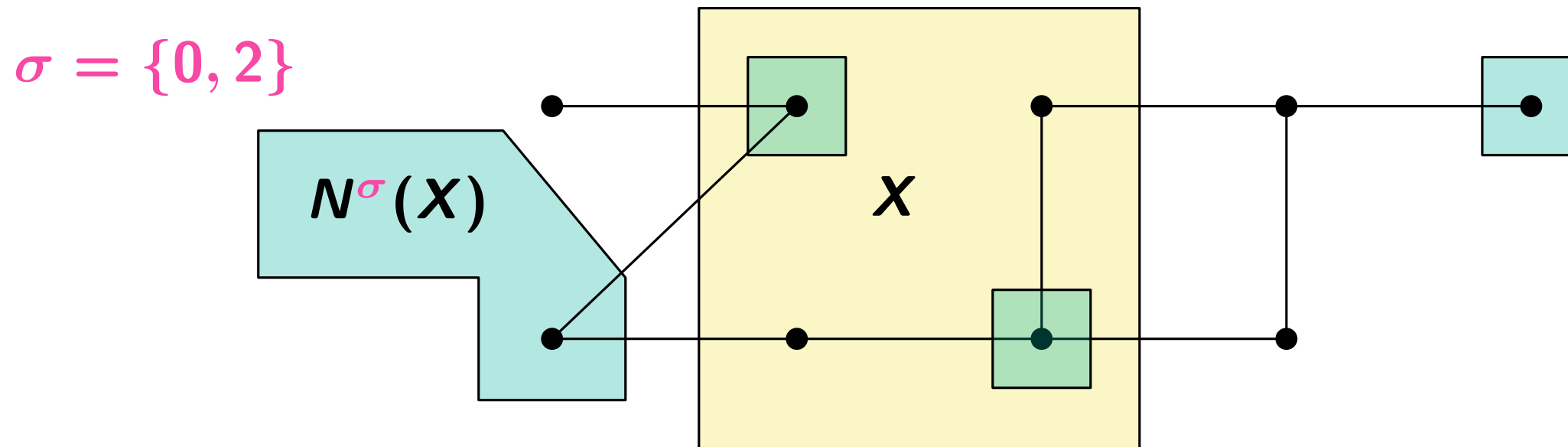
NEO logics: neighborhood operators

- $N(X)$ = all the vertices adjacent to at least one vertex of X
- $N(X, Y)$ = all the vertices adjacent to at least one vertex of X via an edge in Y



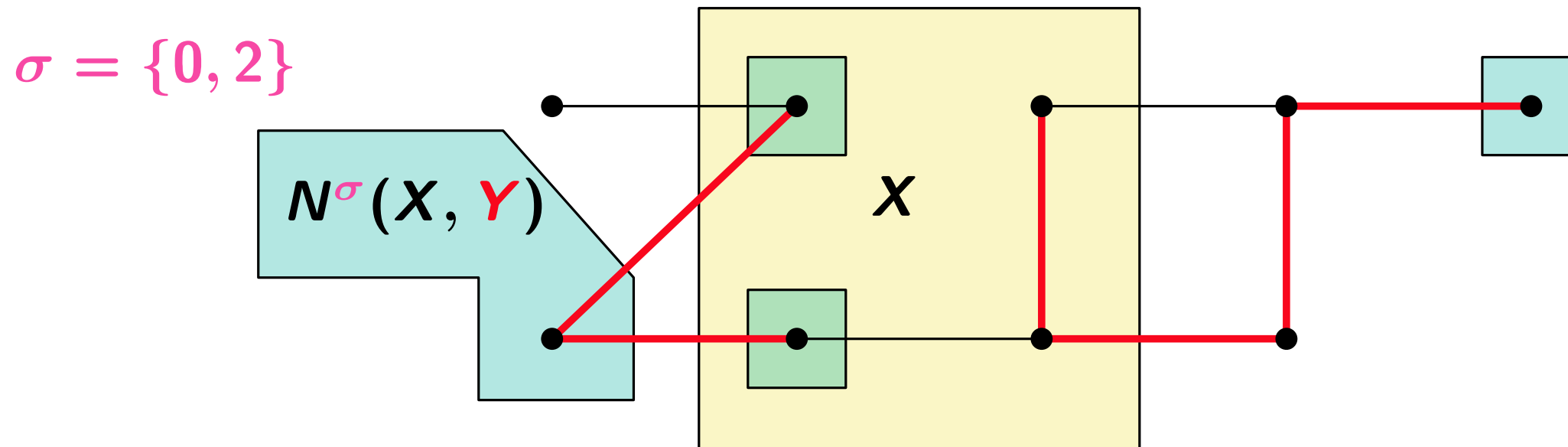
NEO logics: neighborhood operators

- $N(X)$ = all the vertices adjacent to at least one vertex of X
- $N(X, Y)$ = all the vertices adjacent to at least one vertex of X via an edge in Y
- $N^\sigma(X)$ = all the vertices v with $|N(v) \cap X| \in \sigma$



NEO logics: neighborhood operators

- $N(X)$ = all the vertices adjacent to at least one vertex of X
- $N(X, Y)$ = all the vertices adjacent to at least one vertex of X via an edge in Y
- $N^\sigma(X)$ = all the vertices v with $|N(v) \cap X| \in \sigma$
- $N^\sigma(X, Y)$ = all the vertices v with $|N(v, Y) \cap X| \in \sigma$



NEO logics: edge terms

Edge terms:

- Edge set variables

NEO logics: edge terms

Edge terms:

- Edge set variables
- Edge colors, $\emptyset$ and E

NEO logics: edge terms

Edge terms:

- Edge set variables
- Edge colors, $\emptyset$ and E
- $\bar{t}_1$, $t_1 \cap t_2$, $t_1 \cup t_2$ and $t_1 \setminus t_2$

NEO logics: edge terms

Edge terms:

- Edge set variables
- Edge colors, $\emptyset$ and E
- $\bar{t}_1$, $t_1 \cap t_2$, $t_1 \cup t_2$ and $t_1 \setminus t_2$

Example: $(X \cup Y) \cap (E \setminus F)$

NEO logics: **vertex terms**

Vertex terms:

- Vertex set variables

NEO logics: **vertex terms**

Vertex terms:

- Vertex set variables
- Vertex colors, $\emptyset$ and v

NEO logics: vertex terms

Vertex terms:

- Vertex set variables
- Vertex colors, $\emptyset$ and V
- $N^\sigma(t_v)$ and $N^\sigma(t_v, t_e)$ with t_v a vertex term and t_e an edge term

NEO logics: vertex terms

Vertex terms:

- Vertex set variables
- Vertex colors, $\emptyset$ and V
- $N^{\sigma}(t_v)$ and $N^{\sigma}(t_v, t_e)$ with t_v a vertex term and t_e an edge term
- $\bar{t}_1$, $t_1 \cap t_2$, $t_1 \cup t_2$ and $t_1 \setminus t_2$

NEO logics: vertex terms

Vertex terms:

- Vertex set variables
- Vertex colors, $\emptyset$ and V
- $N^{\sigma}(t_v)$ and $N^{\sigma}(t_v, t_e)$ with t_v a vertex term and t_e an edge term
- $\bar{t}_1$, $t_1 \cap t_2$, $t_1 \cup t_2$ and $t_1 \setminus t_2$

Example: $N^{\{1,2,3\}}(N(X), Y) \cap (V \setminus \bar{B})$

Definition of NEO₂

NEO₂ = extension of Existential MSO₂ with:

- **Terms comparisons:** $t_1 \subseteq t_2$ and $t_1 = t_2$

Definition of NEO₂

NEO₂ = extension of Existential MSO₂ with:

- **Terms comparisons:** $t_1 \subseteq t_2$ and $t_1 = t_2$
- **Size measurements:** $|t| \leq m$, $|t| \geq m$, and $|t| = m$

Definition of NEO_2

NEO_2 = extension of **Existential MSO_2** with:

- **Terms comparisons:** $t_1 \subseteq t_2$ and $t_1 = t_2$
- **Size measurements:** $|t| \leq m$, $|t| \geq m$, and $|t| = m$

Example: $\exists X, |X| = k \wedge X \cap N^{\mathbb{N}^+}(X) = \emptyset$

NEO logics: restrictions on σ 's

$\text{NEO}_2[\text{FcoF}] = \text{NEO}_2$ where every σ is **finite or co-finite**
 $\rightarrow \{0\}, \{1, 2, 3, 4, 5\}, \{4, 5, 6, \dots\}, \dots$

NEO logics: restrictions on σ 's

NEO₂[FcoF] = NEO₂ where every σ is **finite or co-finite**

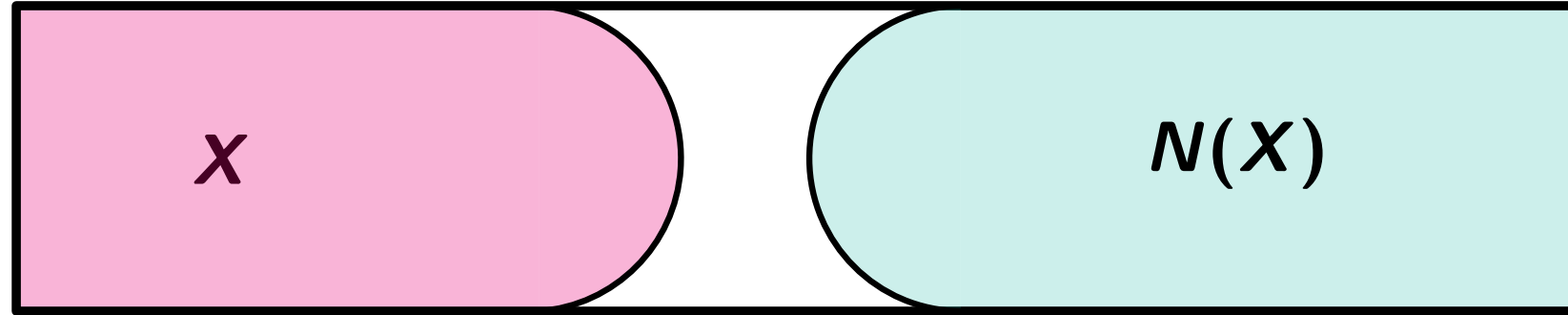
→ {0}, {1, 2, 3, 4, 5}, {4, 5, 6, ...}, ...

NEO₂[FRec] = NEO₂ where every σ is **finitetely recognizable**

→ {even number}, $\{i \in \mathbb{N} \mid i \equiv 3 \text{ modulo } 5\}$, {1, 2, 3, 4, 5, 10, 20, 30, ...}, ...

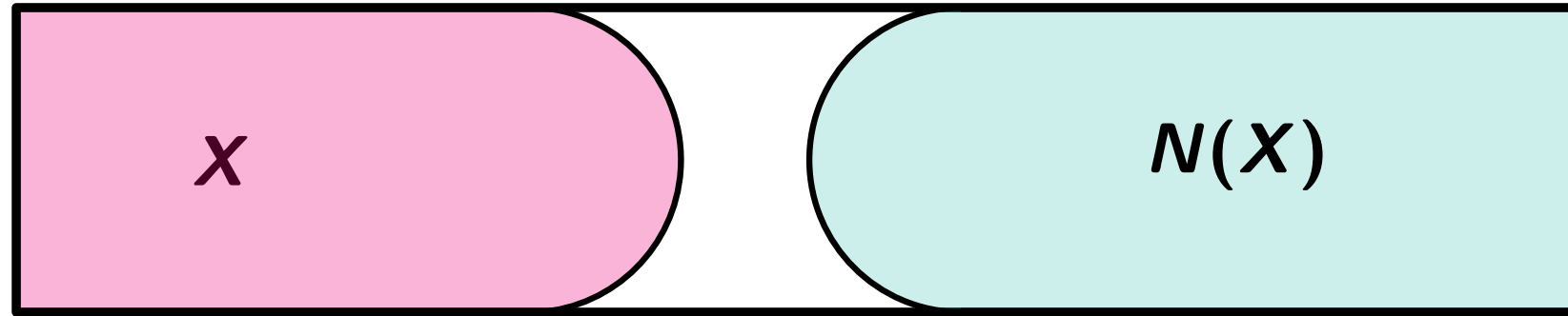
Basic examples

k-Independent Set: $\exists X, |X| \geq k \wedge X \cap N(X) = \emptyset$



Basic examples

k -Independent Set: $\exists X, |X| \geq k \wedge X \cap N(X) = \emptyset$

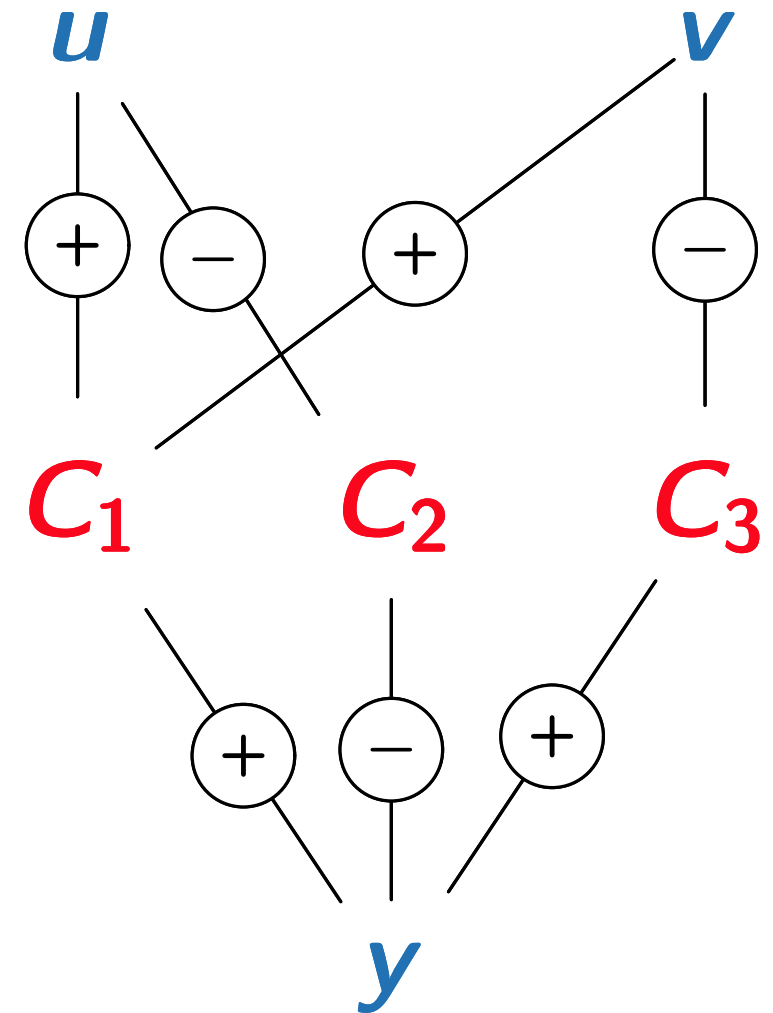


k -Dominating Set: $\exists X, |X| \leq k \wedge X \cup N(X) = V$



CNF SAT

SAT: $\exists X \subseteq \bar{c}, N(X, +) \cup N(\bar{X}, -) = c$

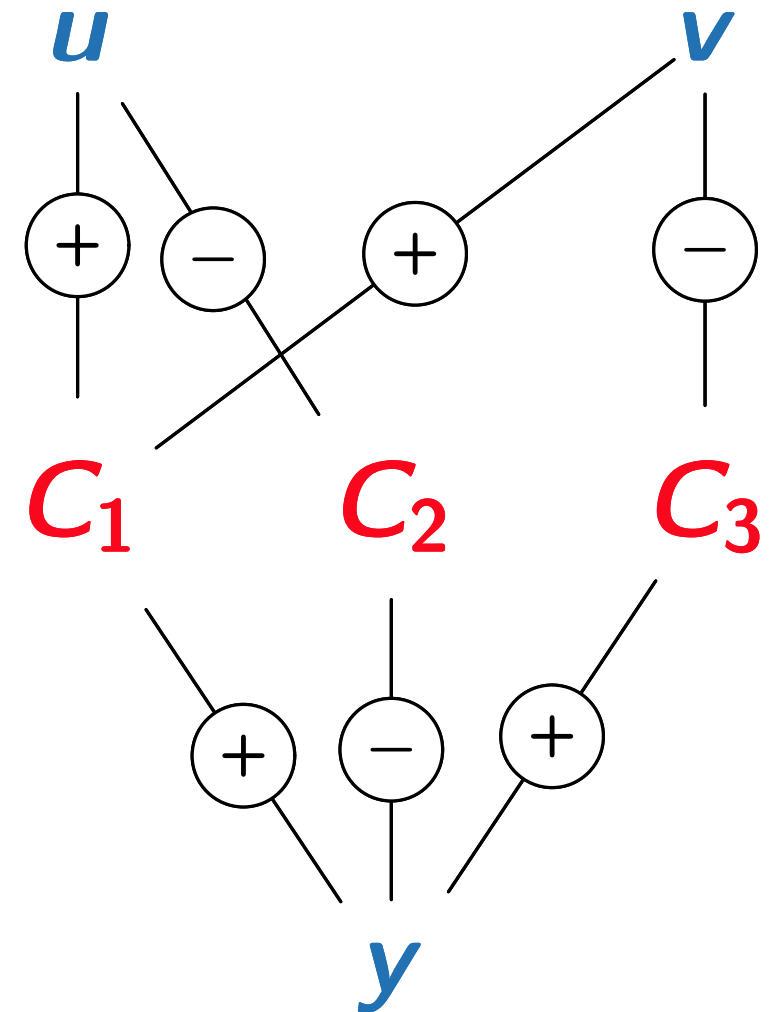


CNF SAT

SAT: $\exists X \subseteq \bar{c}, N(X, +) \cup N(\bar{X}, -) = c$

Max-SAT: maximize $|Y|$

$\exists X \subseteq \bar{c}, \exists Y \subseteq c, N(X, +) \cup N(\bar{X}, -) = Y$



Acyclicity and connectivity

$L + \text{ACK}$ = extension of L with:

- $\text{acy}(t, Y)$ true when the subgraph (t, Y) is a forest

Acyclicity and connectivity

$L + ACK$ = extension of L with:

- $acy(t, Y)$ true when the subgraph (t, Y) is a forest
- $con(t, Y)$ true when the subgraph (t, Y) is connected

Acyclicity and connectivity

$L + ACK$ = extension of L with:

- $acy(t, Y)$ true when the subgraph (t, Y) is a **forest**
- $con(t, Y)$ true when the subgraph (t, Y) is **connected**
- $clique(t)$ true when the subgraph induced by t is a **clique**

Acyclicity and connectivity

L + ACK = extension of **L** with:

- **acy**(t, Y) true when the subgraph (t, Y) is a **forest**
- **con**(t, Y) true when the subgraph (t, Y) is **connected**
- **clique**(t) true when the subgraph induced by t is a **clique**

Hamiltonian Cycle: $\exists Y \wedge \text{con}(V, Y) \wedge N^{\{2\}}(V, Y) = V$

Model Checking: tree-width

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+AC** are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

Model Checking: tree-width

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+AC** are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

[B., Dreier, Jaffke, SODA 2023]

ECML is equivalent semantically to **NEO₂[FRec]**

Model Checking: tree-width

[Pilipczuk, MFCS 2011]

All problems expressible in **ECML+AC** are decidable in time $2^{\mathcal{O}(\text{tw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$

[B., Dreier, Jaffke, SODA 2023]

ECML is equivalent semantically to **NEO₂[FRec]**

Optimal under ETH

$\Rightarrow$ no algorithm in time $2^{\mathcal{O}(\text{tw})} n^{\mathcal{O}(1)}$ for **Max. Independent Set**

Model Checking: other widths

NEO₁ = NEO₂ without edge terms and edge variables

Model Checking: other widths

NEO₁ = NEO₂ without edge terms and edge variables

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

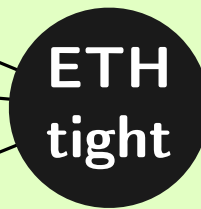
- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$

Model Checking: other widths

NEO₁ = NEO₂ without edge terms and edge variables

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$
- 
- [B. Korhonen, Nederlof, STACS 2023]
- [B., Hamm, Jaffke, Lima, ESA 2024]

Model Checking: other widths

NEO₁ = NEO₂ without edge terms and edge variables

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$

- First **logic-based** algorithmic meta-theorem for **mim-width**

Model Checking: other widths

NEO₁ = NEO₂ without edge terms and edge variables

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$
-
- First **logic-based** algorithmic meta-theorem for **mim-width**
 - For **mim-width**, we can even evaluate neighborhood operators in **graph powers**

Ingredients of the proofs

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$

Ingredients of the proofs

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$
- One classic DP algorithms

Ingredients of the proofs

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - rank-width: $2^{\mathcal{O}(\text{rw}^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - $\mathbb{Q}$ -rank-width: $\text{rw}_{\mathbb{Q}}^{\mathcal{O}(\text{rw}_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - mim-width: $n^{\mathcal{O}(\text{mim} \cdot |\varphi|)}$
-
- One classic DP algorithms
 - Two partial interpretations α, β over $\mathbf{A} \subseteq \mathbf{V}(\mathbf{G})$ are φ -equivalent if $N^\sigma(\alpha(\mathbf{X})) \setminus \mathbf{A} = N^\sigma(\beta(\mathbf{X})) \setminus \mathbf{A}$ for every $N^\sigma(\mathbf{X})$ in φ
→ nb. of equivalence classes bounded by $(d+1)^{\text{cw}}, 2^{d \cdot \text{rw}^2}, 2^{d \cdot \text{rw}_{\mathbb{Q}}} \log \text{rw}_{\mathbb{Q}}$ and $n^{d \cdot \text{mim}}$

Ingredients of the proofs

[B., Dreier, Jaffke, SODA 2023]

All problems expressible in **NEO₁[FcoF]+AC** are decidable in time (simplified):

- clique-width: $2^{\mathcal{O}(cw \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - rank-width: $2^{\mathcal{O}(rw^2 \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - $\mathbb{Q}$ -rank-width: $rw_{\mathbb{Q}}^{\mathcal{O}(rw_{\mathbb{Q}} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
 - mim-width: $n^{\mathcal{O}(mim \cdot |\varphi|)}$
-
- One classic DP algorithms
 - Two partial interpretations α, β over $A \subseteq V(G)$ are φ -equivalent if $N^\sigma(\alpha(X)) \setminus A = N^\sigma(\beta(X)) \setminus A$ for every $N^\sigma(X)$ in φ
→ nb. of equivalence classes bounded by $(d+1)^{cw}$, $2^{d \cdot rw^2}$, $2^{d \cdot rw_{\mathbb{Q}} \log rw_{\mathbb{Q}}}$ and $n^{d \cdot mim}$
 - Rank-based approach for connectivity and a simple reduction for acyclicity

What about FRec instead of FcoF?

[B., Dreier, Jaffke, SODA 2023]

The model checking problem for $\text{NEO}_1[\text{FRec}] + \text{AC}$ is:

- clique-width: $2^{\mathcal{O}(\text{cw} \cdot |\varphi|)} n^{\mathcal{O}(1)}$
- rank-width: FPT
- $\mathbb{Q}$ -rank-width: FPT
- mim-width: **NP-hard** on interval graphs even for $\text{NEO}_1[\text{Parity}]$

Space complexity

Problem: $2^{\mathcal{O}(tw)} n^{\mathcal{O}(1)}$ time algorithms use $2^{\mathcal{O}(tw)} n^{\mathcal{O}(1)}$ space!

Space complexity

Problem: $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ time algorithms use $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ space!

Can we reduce the space complexity and keep the running time?

Space complexity

Problem: $2^{\mathcal{O}(tw)} n^{\mathcal{O}(1)}$ time algorithms use $2^{\mathcal{O}(tw)} n^{\mathcal{O}(1)}$ space!

Can we reduce the space complexity and keep the running time?

[Lokshtanov, Marx, Saurabh, SODA 2011]

Many problems with a $2^{\mathcal{O}(tw)} n^{\mathcal{O}(1)}$ time algorithm can be solved in time $2^{\mathcal{O}(tw^2)} n^{\mathcal{O}(1)}$ and **polynomial space**

Space complexity

Problem: $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ time algorithms use $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ space!

Can we reduce the space complexity and keep the running time?

[Lokshtanov, Marx, Saurabh, SODA 2011]

Many problems with a $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ time algorithm can be solved in time $2^{\mathcal{O}(t_w^2)} n^{\mathcal{O}(1)}$ and **polynomial space**

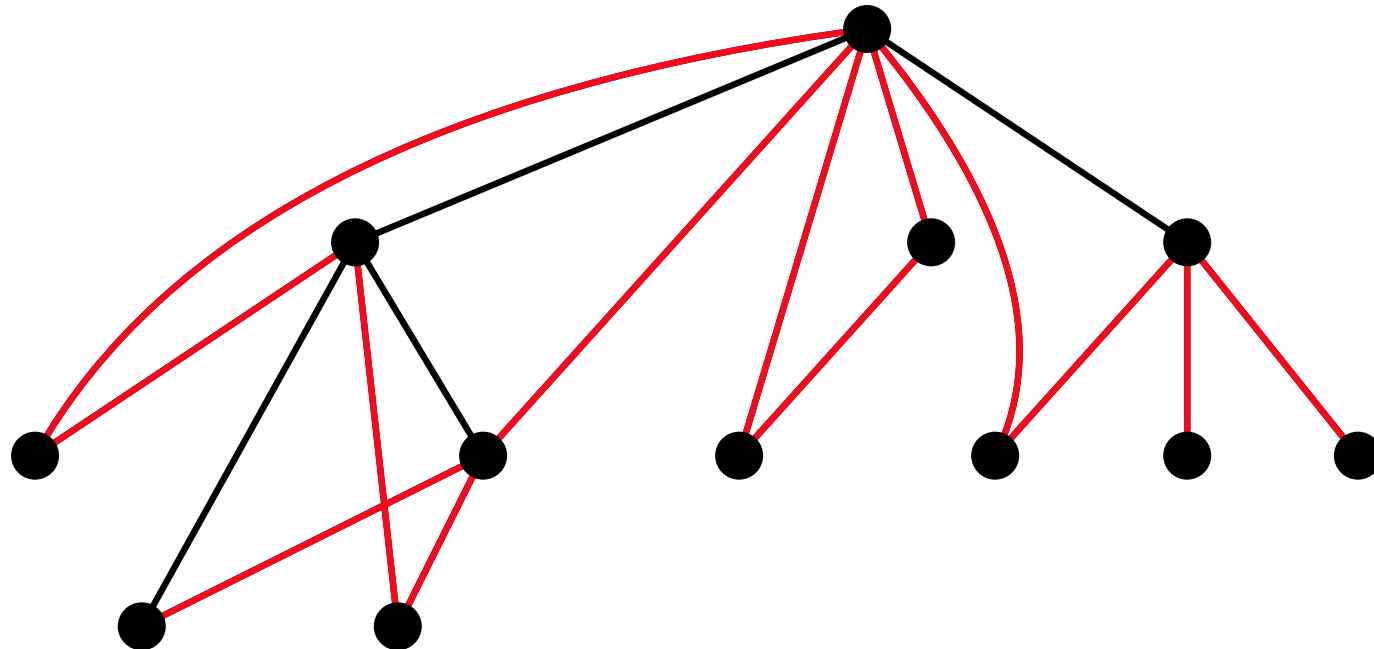
[Pilipczuk, Wrochna, TOCT 2018]

There is no $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ time algorithm for 3-Coloring using $2^{\mathcal{O}(t_w)} n^{\mathcal{O}(1)}$ space unless some reasonable conjecture on **Longest Common Subsequence** is false

Solution: Tree-depth

[Něsetřil and Ossona de Mendez, EJC 2006]

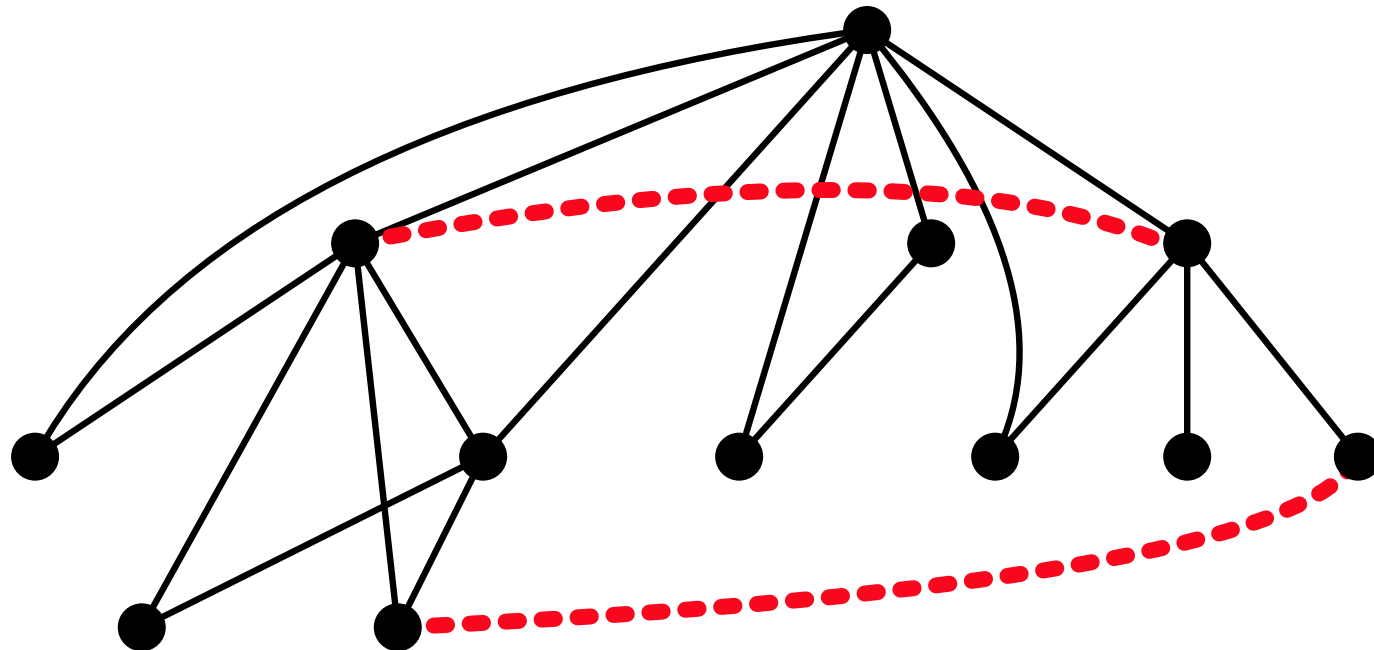
Tree-depth of a graph G = minimum depth of a tree T with $V(T) = V(G)$ such that $E(G) \subseteq$ the **ancestor-descendant relation of T**



Solution: Tree-depth

[Něsetřil and Ossona de Mendez, EJC 2006]

Tree-depth of a graph G = minimum depth of a tree T with $V(T) = V(G)$ such that $E(G) \subseteq$ the **ancestor-descendant relation** of T

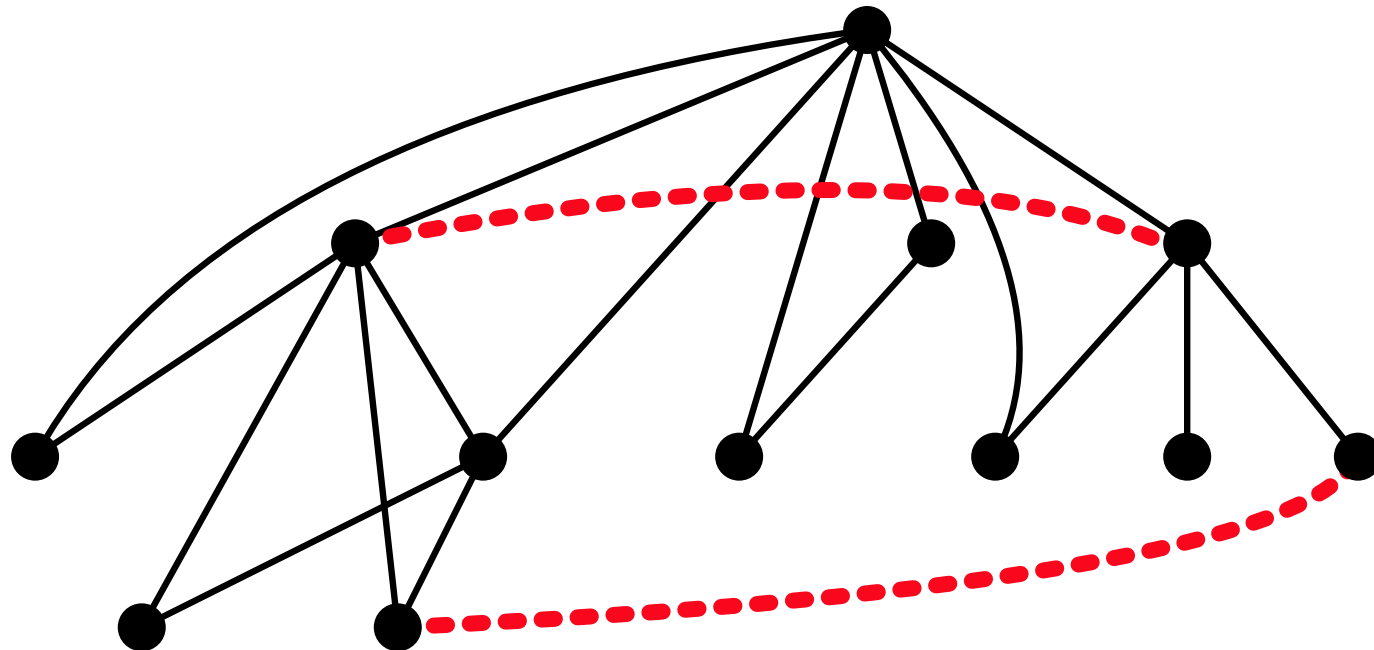


Solution: Tree-depth

[Něsetřil and Ossona de Mendez, EJC 2006]

Tree-depth of a graph G = minimum depth of a tree T with $V(T) = V(G)$ such that $E(G) \subseteq$ the **ancestor-descendant relation** of T

T is an **elimination tree** of G



Tree-depth facts

[Bodlaender, Gilbert, Hafsteinsson, Kloks, J. Algo. 1995]

For all graph G , we have $\text{tw}(G) \leq \text{td}(G) \leq \text{tw}(G) \cdot \log n$

Tree-depth facts

[Bodlaender, Gilbert, Hafsteinsson, Kloks, J. Algo. 1995]

For all graph G , we have $\text{tw}(G) \leq \text{td}(G) \leq \text{tw}(G) \cdot \log n$

[Nadara, Pilipczuk, and Smulewicz, ESA 2022]

An optimal elimination tree can be computed in time $2^{\mathcal{O}(\text{td}^2)} \cdot n^{\mathcal{O}(1)}$

Tree-depth facts

[Bodlaender, Gilbert, Hafsteinsson, Kloks, J. Algo. 1995]

For all graph G , we have $\text{tw}(G) \leq \text{td}(G) \leq \text{tw}(G) \cdot \log n$

[Nadara, Pilipczuk, and Smulewicz, ESA 2022]

An optimal elimination tree can be computed in time $2^{\mathcal{O}(\text{td}^2)} \cdot n^{\mathcal{O}(1)}$

- Can we approximate tree-depth in time $2^{\mathcal{O}(\text{td}^2)} \cdot n^{\mathcal{O}(1)}$?

Tree-depth: algorithmic applications

Many problems can be solved in time $2^{\mathcal{O}(\text{td})}n$ and **polynomial space** $= n^{\mathcal{O}(1)}$ given an elimination tree of depth td

- Independent Set **[Folklore]**

Tree-depth: algorithmic applications

Many problems can be solved in time $2^{\mathcal{O}(\text{td})}n$ and **polynomial space** $= n^{\mathcal{O}(1)}$ given an elimination tree of depth td

- Independent Set [\[Folklore\]](#)
- CSP [\[Freuder, Quinn, IJCT 1985\]](#)

Tree-depth: algorithmic applications

Many problems can be solved in time $2^{\mathcal{O}(\text{td})}n$ and **polynomial space** $= n^{\mathcal{O}(1)}$ given an elimination tree of depth td

- Independent Set [\[Folklore\]](#)
- CSP [\[Freuder, Quinn, IJCT 1985\]](#)
- #perfect matching, set covering and partitioning pb. [\[Fürier, Yi, TCS 2017\]](#)

Tree-depth: algorithmic applications

Many problems can be solved in time $2^{\mathcal{O}(\text{td})}n$ and **polynomial space** $= n^{\mathcal{O}(1)}$ given an elimination tree of depth td

- Independent Set [\[Folklore\]](#)
- CSP [\[Freuder, Quinn, IJCT 1985\]](#)
- #perfect matching, set covering and partitioning pb. [\[Fürier, Yi, TCS 2017\]](#)
- Dominating Set [\[Pilipczuk, Wrochna, TOCT 2018\]](#)
→ space complexity: $\mathcal{O}(\text{td} \cdot \log n)$

Tree-depth: algorithmic applications

Many problems can be solved in time $2^{\mathcal{O}(\text{td})}n$ and **polynomial space** $= n^{\mathcal{O}(1)}$ given an elimination tree of depth td

- Independent Set [\[Folklore\]](#)
- CSP [\[Freuder, Quinn, IJCT 1985\]](#)
- #perfect matching, set covering and partitioning pb. [\[Fürier, Yi, TCS 2017\]](#)
- Dominating Set [\[Pilipczuk, Wrochna, TOCT 2018\]](#)
→ space complexity: $\mathcal{O}(\text{td} \cdot \log n)$
- Connected VC, CDS COCT, FVS, Steiner Tree [\[Hegerfeld, Kratsch, STACS 2020\]](#)

Tree-depth: algorithmic applications

Many problems can be solved in time $2^{\mathcal{O}(\text{td})}n$ and **polynomial space** $= n^{\mathcal{O}(1)}$ given an elimination tree of depth td

- Independent Set [\[Folklore\]](#)
- CSP [\[Freuder, Quinn, IJCT 1985\]](#)
- #perfect matching, set covering and partitioning pb. [\[Fürer, Yi, TCS 2017\]](#)
- Dominating Set [\[Pilipczuk, Wrochna, TOCT 2018\]](#)
→ space complexity: $\mathcal{O}(\text{td} \cdot \log n)$
- Connected VC, CDS COCT, FVS, Steiner Tree [\[Hegerfeld, Kratsch, STACS 2020\]](#)
- Hamiltonian Cycle [\[Nederlof, Pilipczuk, Swennenhuis, Wegrzycki, DM 2023\]](#)

Tree-depth

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

Tree-depth

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

Tree-depth

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

- Generalizes and unifies the previous results and works for new problems like **SAT** and **Max-SAT** via the incidence graph

Tree-depth

[B., Checkan, Stamoulis, SODA 2026] (simplified)

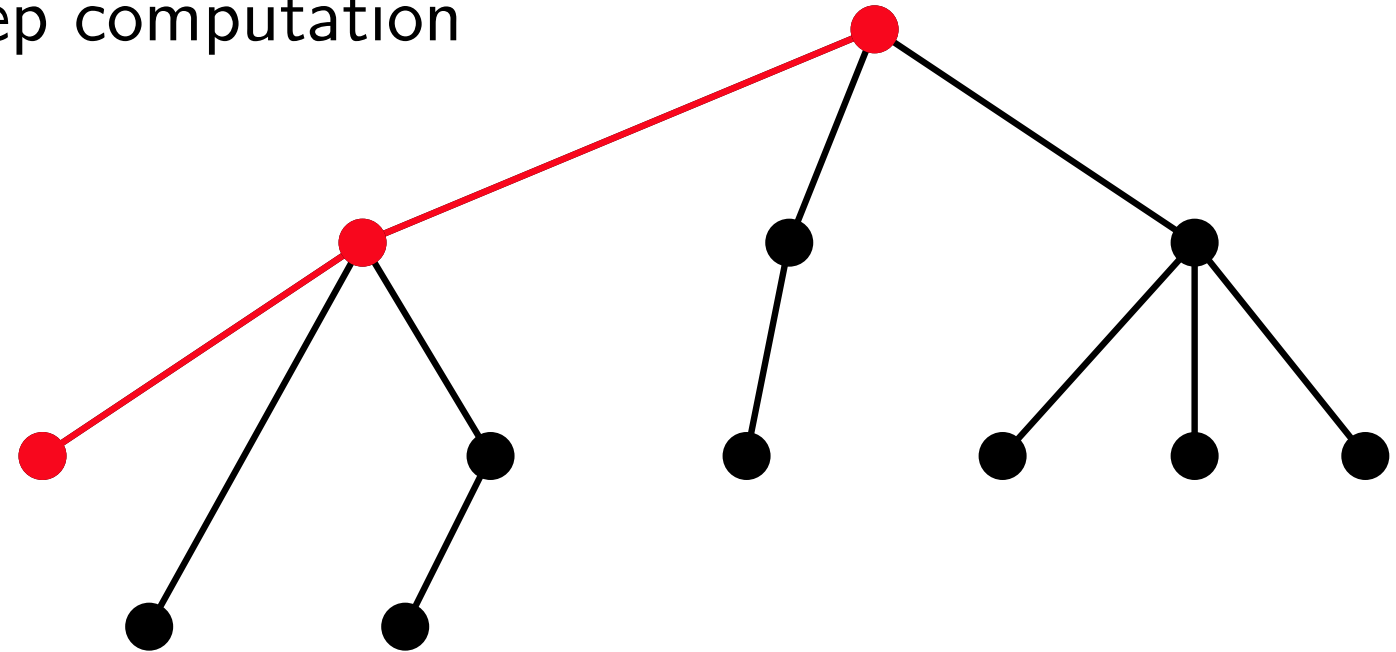
All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

- Generalizes and unifies the previous results and works for new problems like **SAT** and **Max-SAT** via the incidence graph
- We can count the models when all the σ are periodic on $[1, +\infty]$ ($\#SAT$)

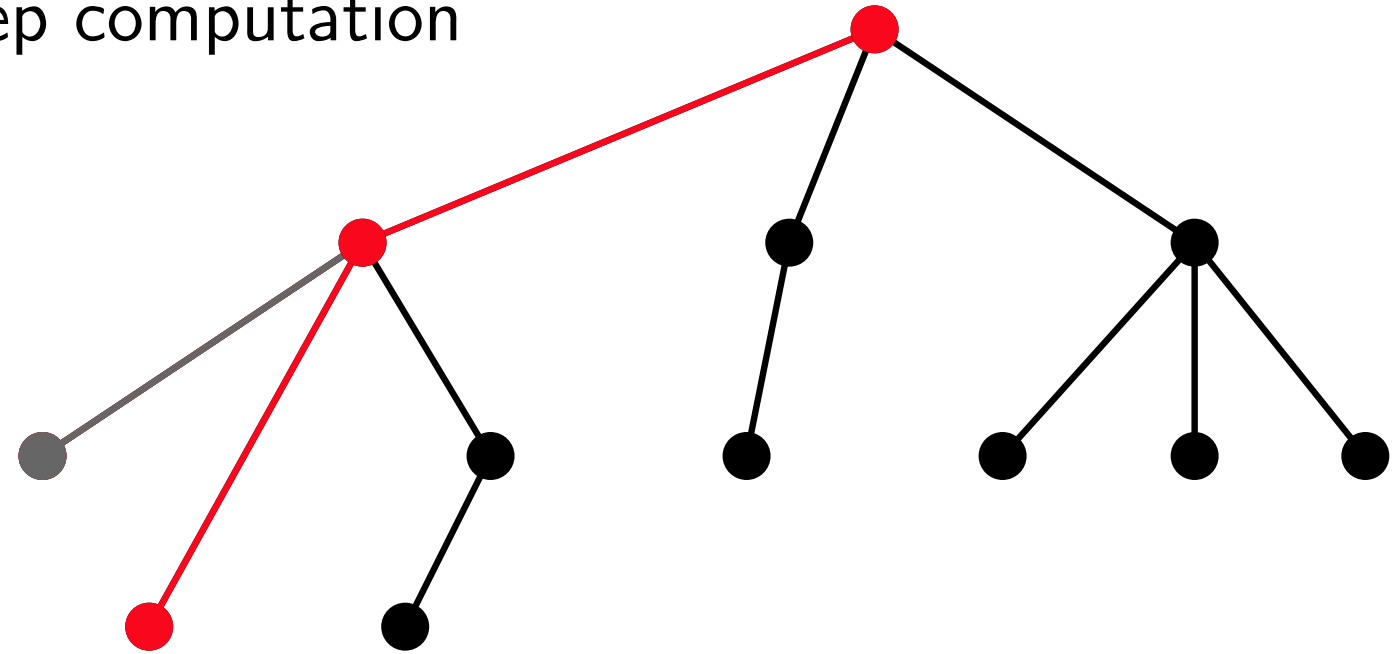
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path



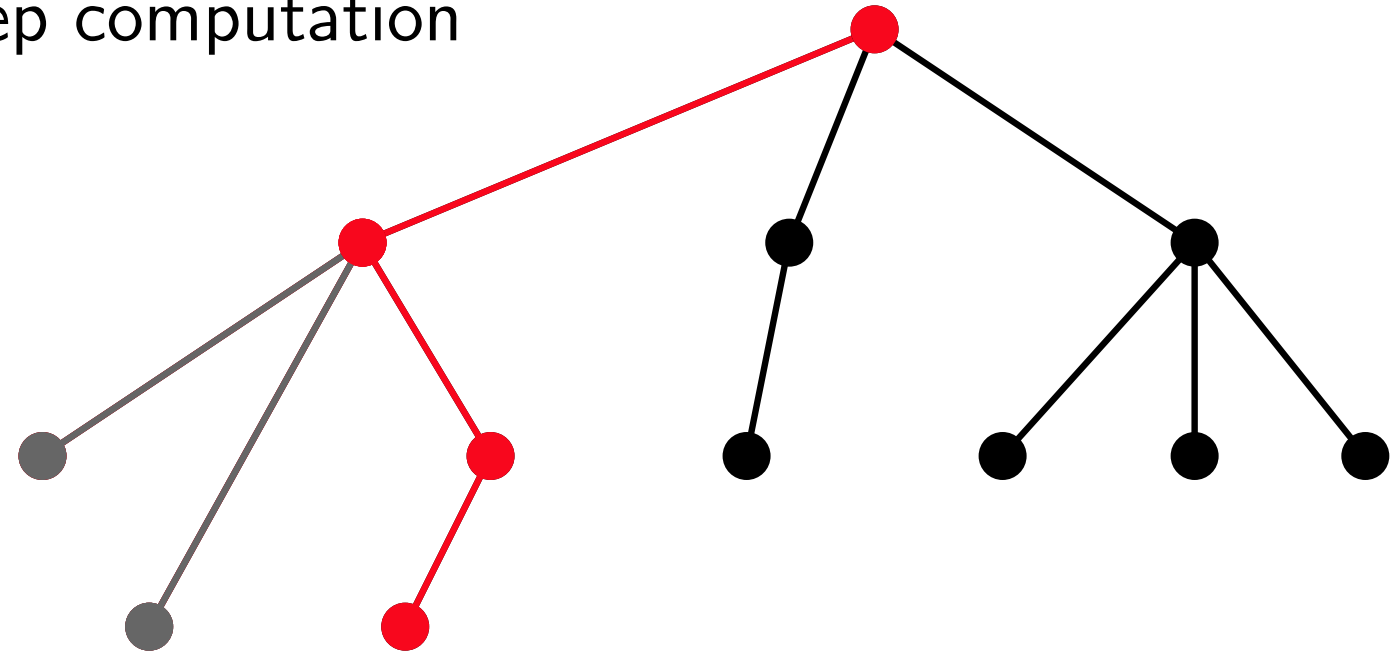
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path



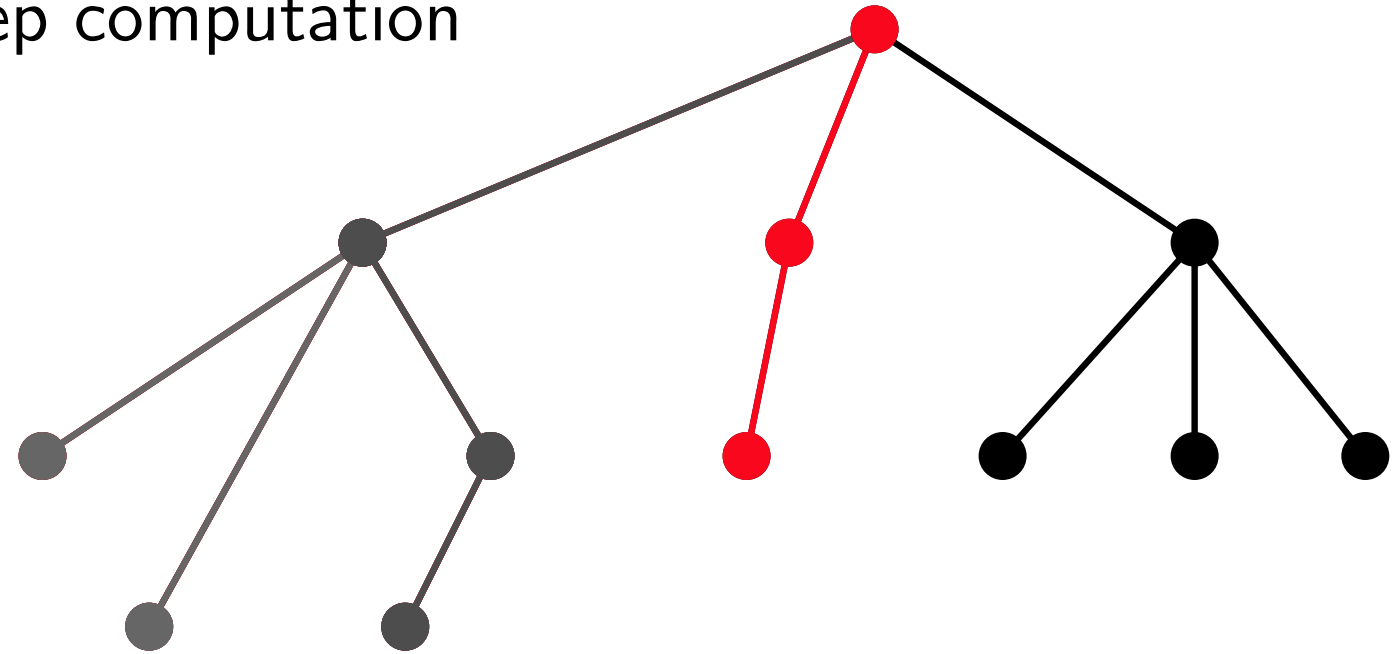
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path



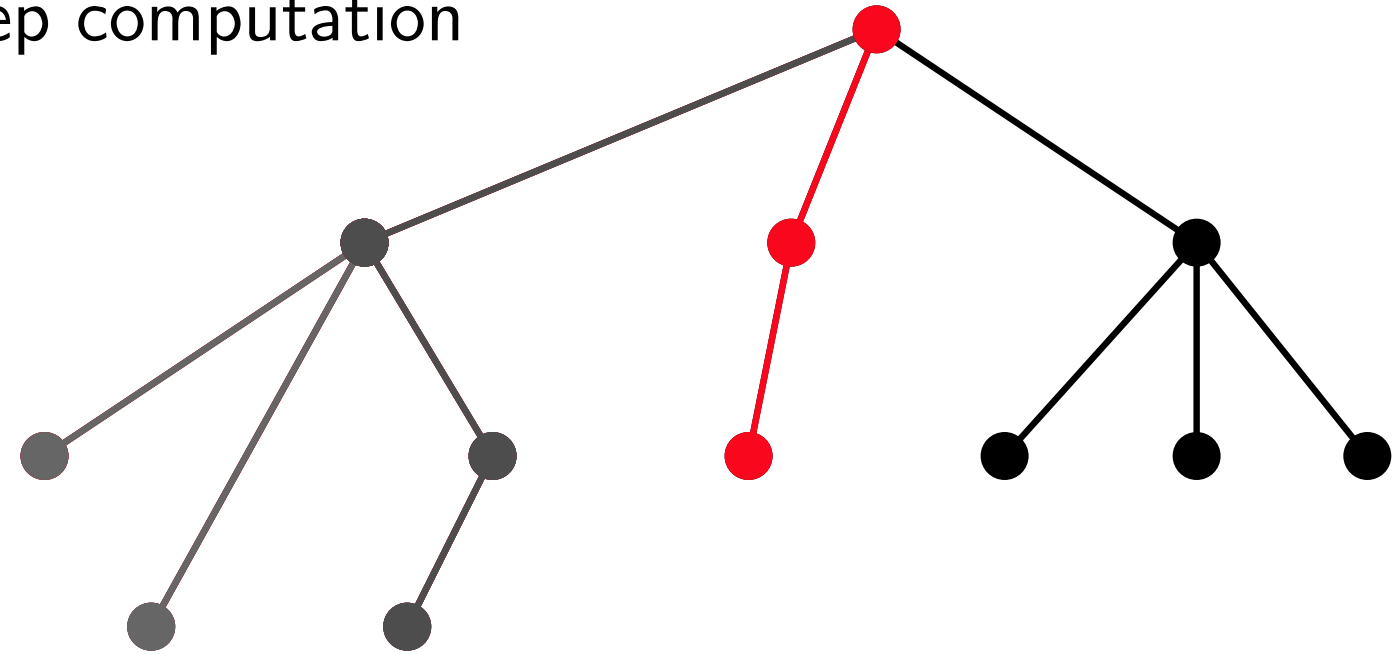
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path



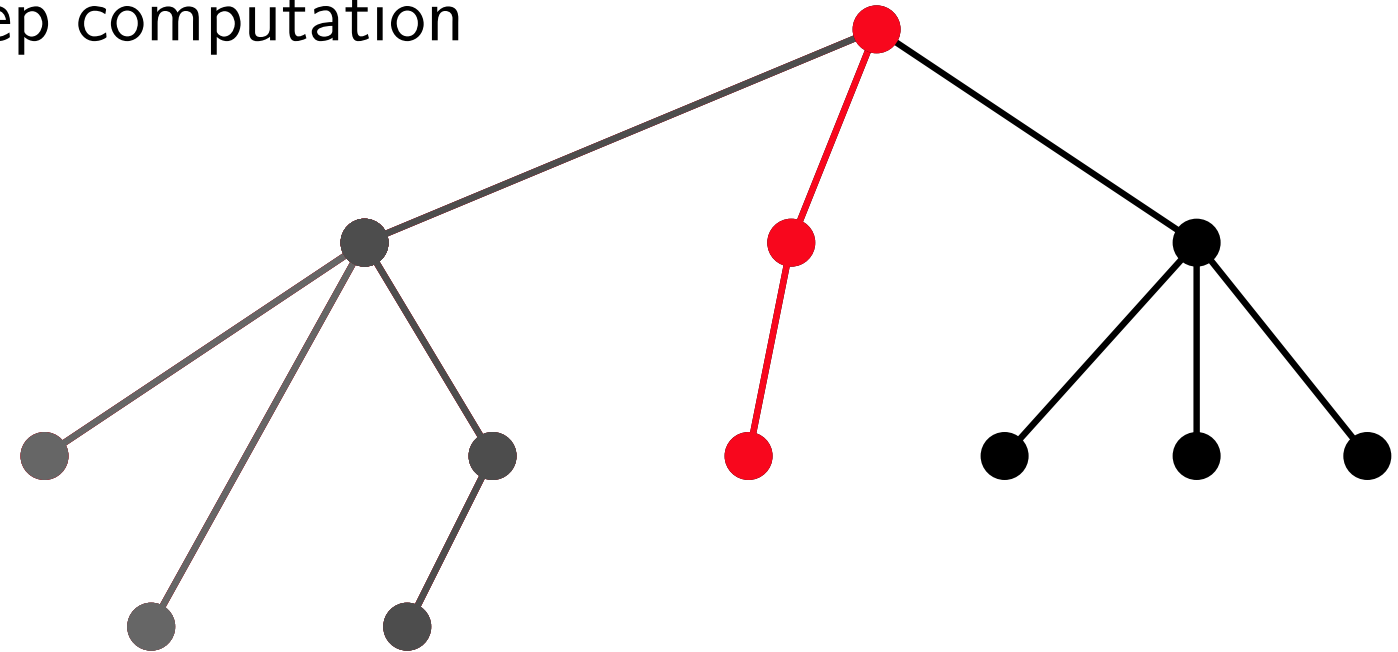
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path
- d_φ the minimum d such that every set σ in φ is periodic on $[d, +\infty]$



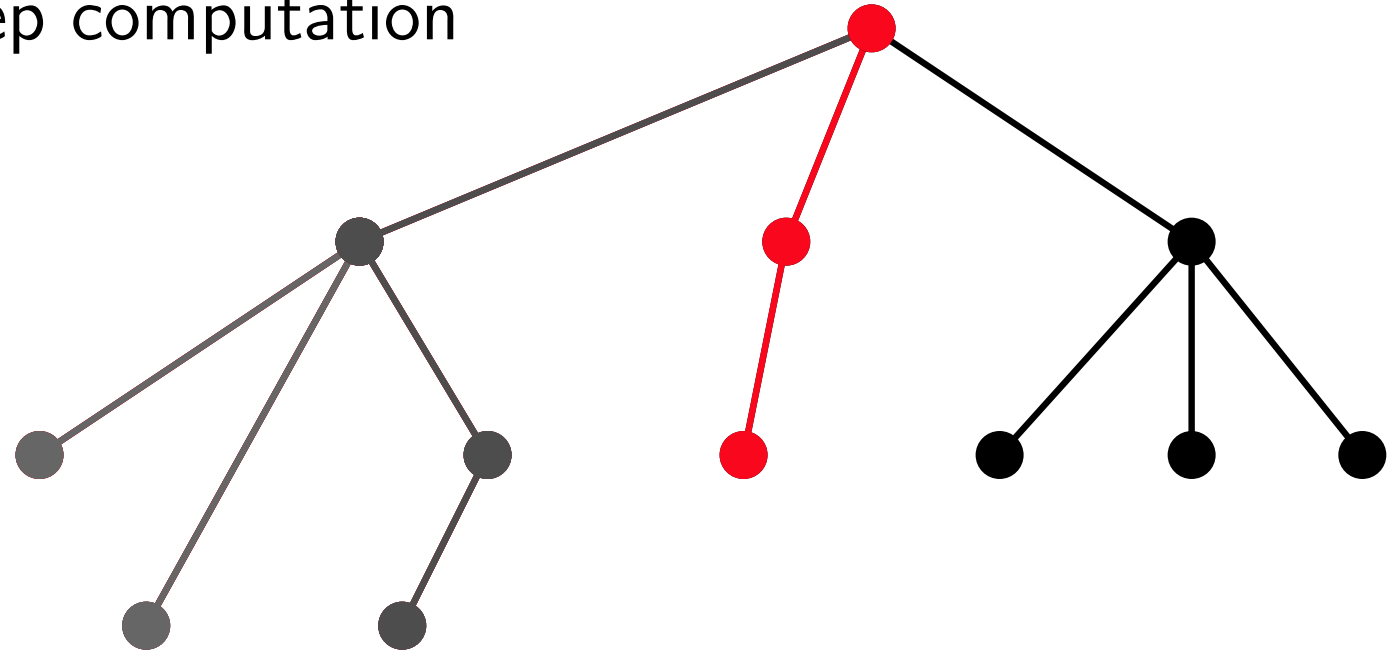
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path
- d_φ the minimum d such that every set σ in φ is periodic on $[d, +\infty]$
- p_φ the l.c.m. of the periods of each set σ in φ



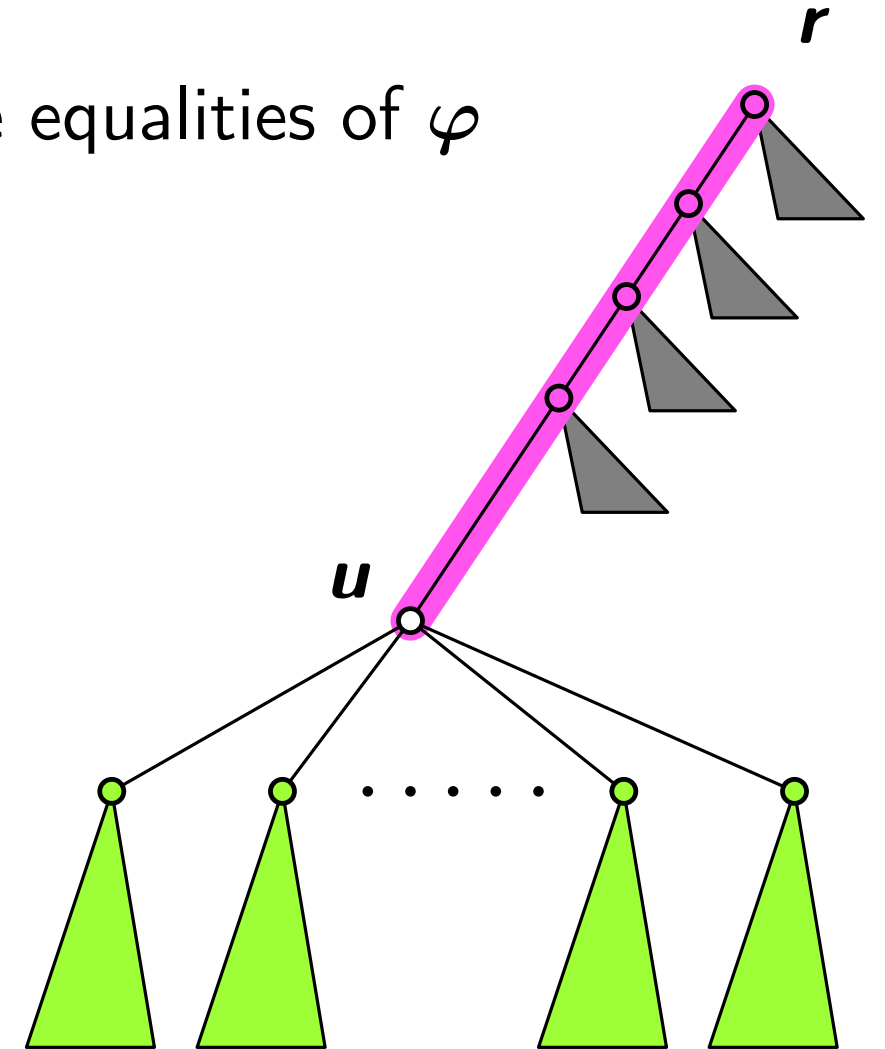
Proof ingredients

- Top-bottom recursive algorithm, we keep computation information only on a rooted path
- d_φ the minimum d such that every set σ in φ is periodic on $[d, +\infty]$
- p_φ the l.c.m. of the periods of each set σ in φ
- We compute 2 tables at each node with at most $((d_\varphi + 1)^2 \cdot p_\varphi)^{|\varphi| \cdot \text{td}} \cdot 2^{|\varphi|}$ indexes
→ we call each entry of every table at most once



Problems and solutions I

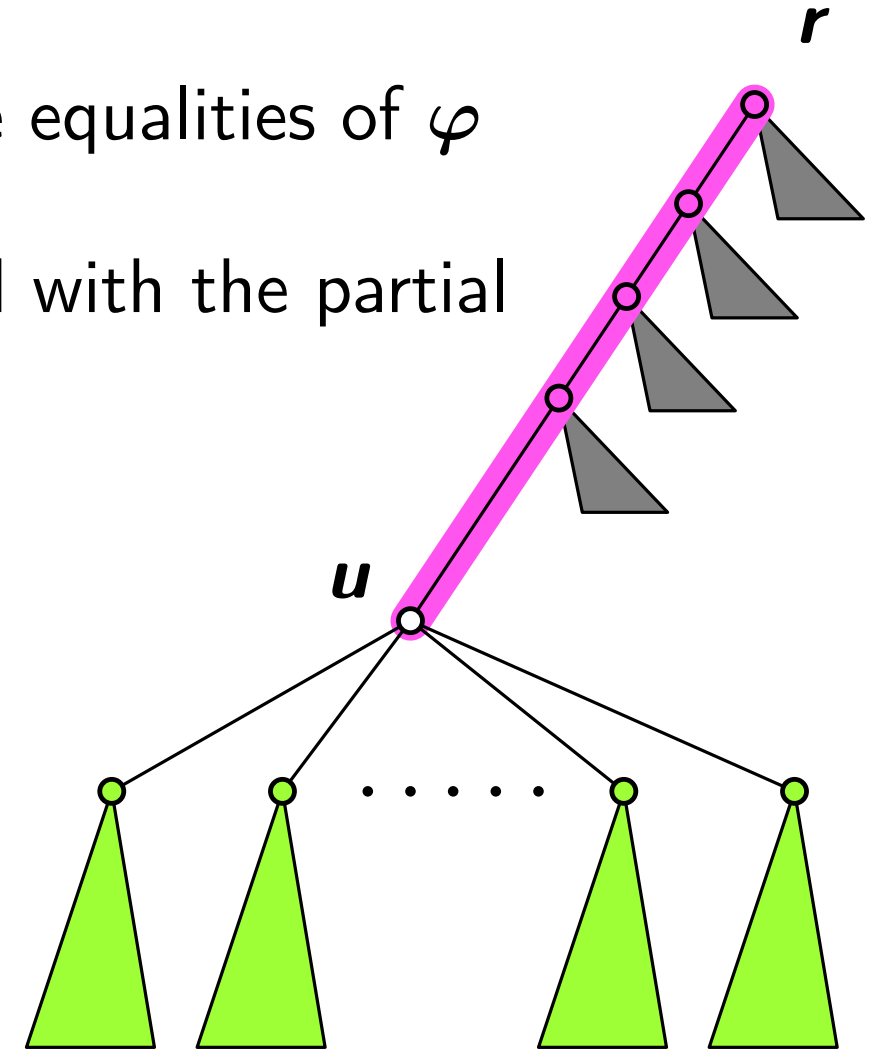
Each index keeps some information about the **tail** and the equalities of φ



Problems and solutions I

Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

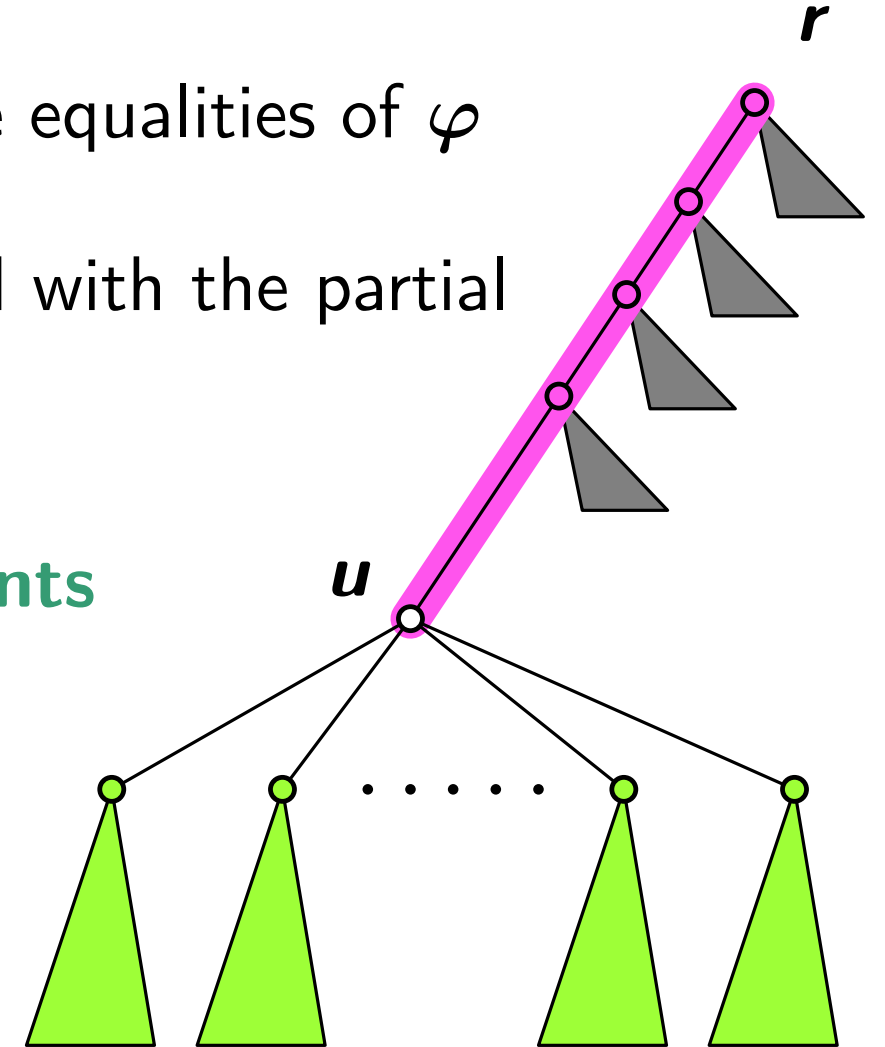


Problems and solutions I

Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

→ Some variables keeps tracks of the ℓ **size measurements**
 $n^{\mathcal{O}(\ell)}$ in the final running time



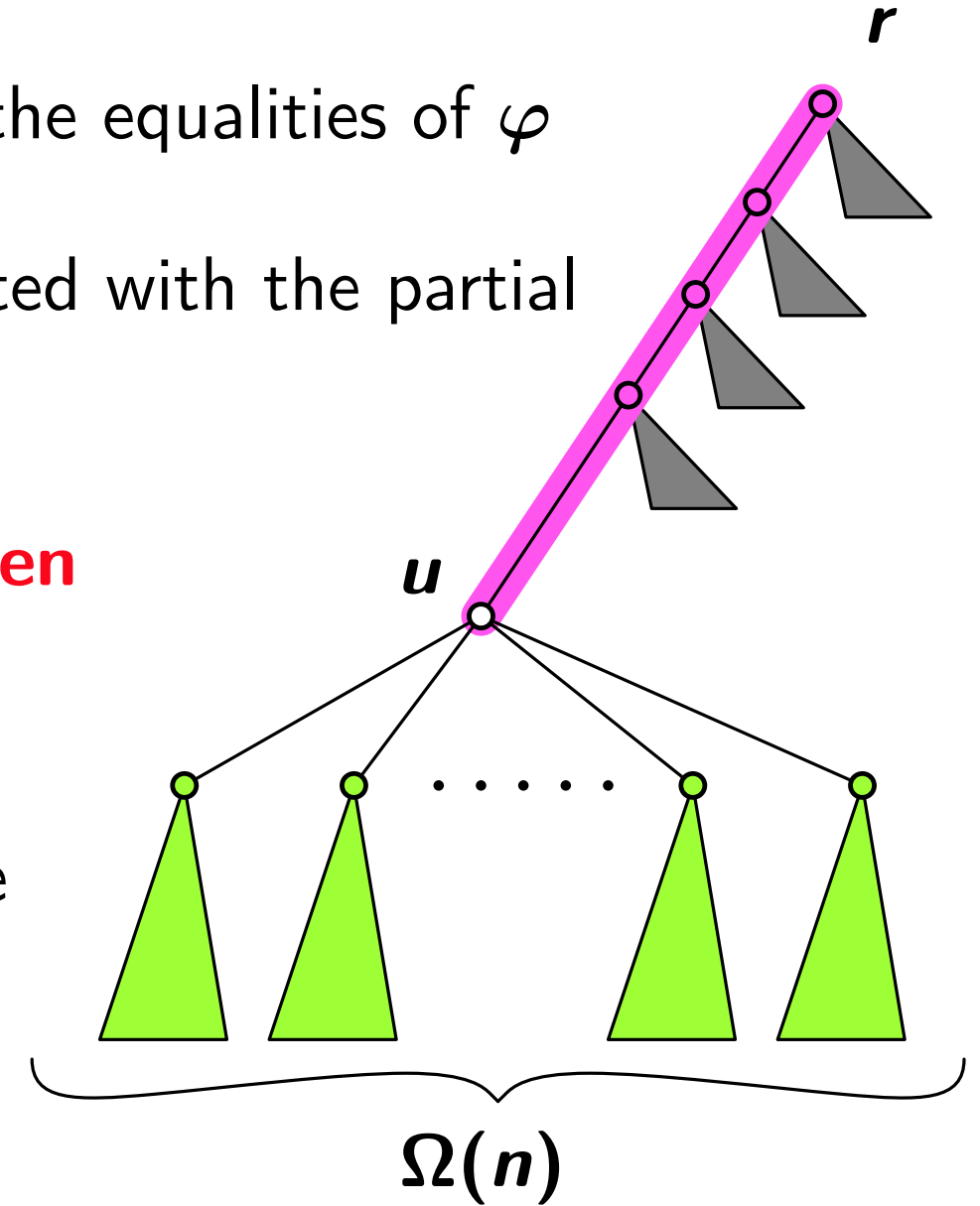
Problems and solutions I

Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

Problem: a node of T can have $\Omega(n)$ children

- DS: we can't keep track of which vertices in **tail** are dominated



Problems and solutions I

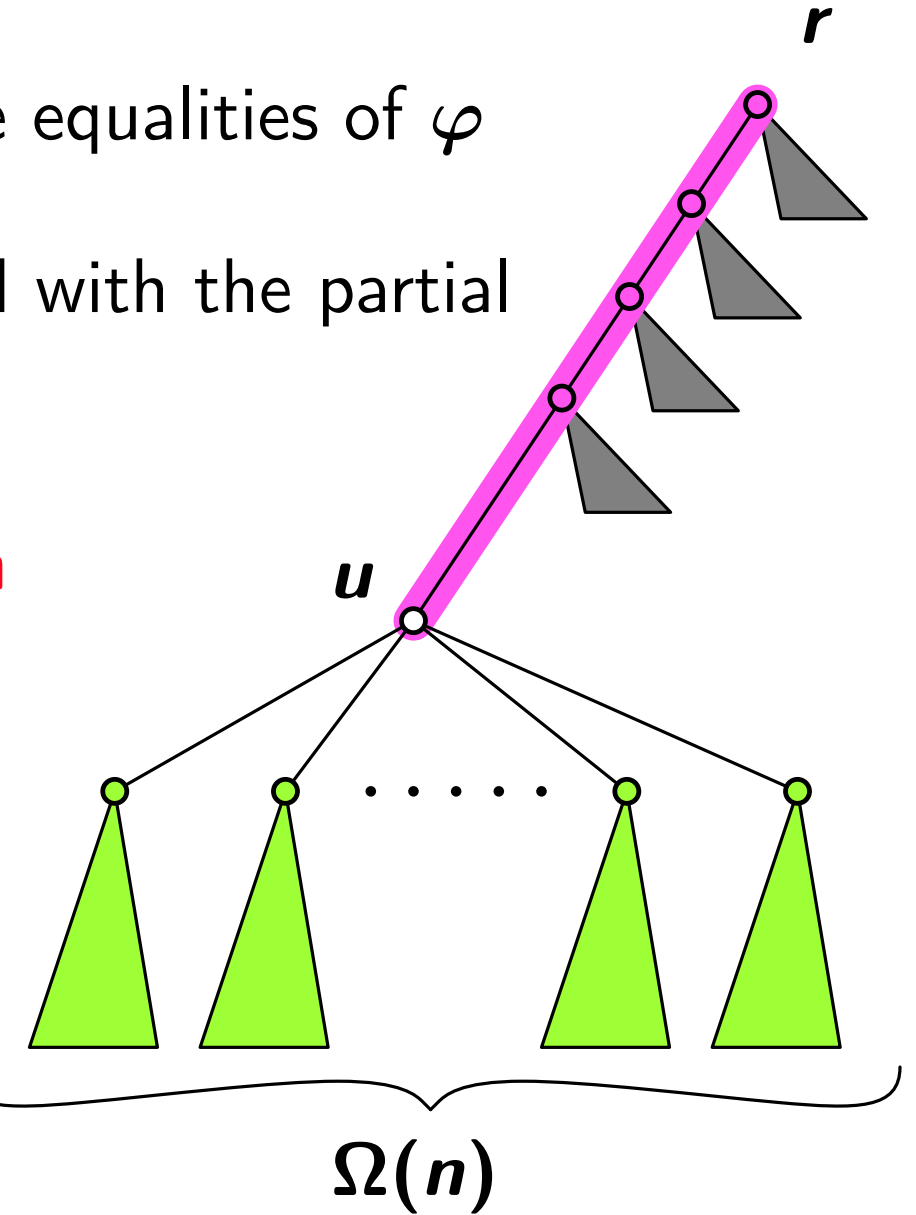
Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

Problem: a node of T can have $\Omega(n)$ children

- DS: we can't keep track of which vertices in **tail** are dominated

→ Solution: $f: \text{tail} \rightarrow \{D, \text{Allowed}, \text{Forbidden}\}$ and inclusion-exclusion



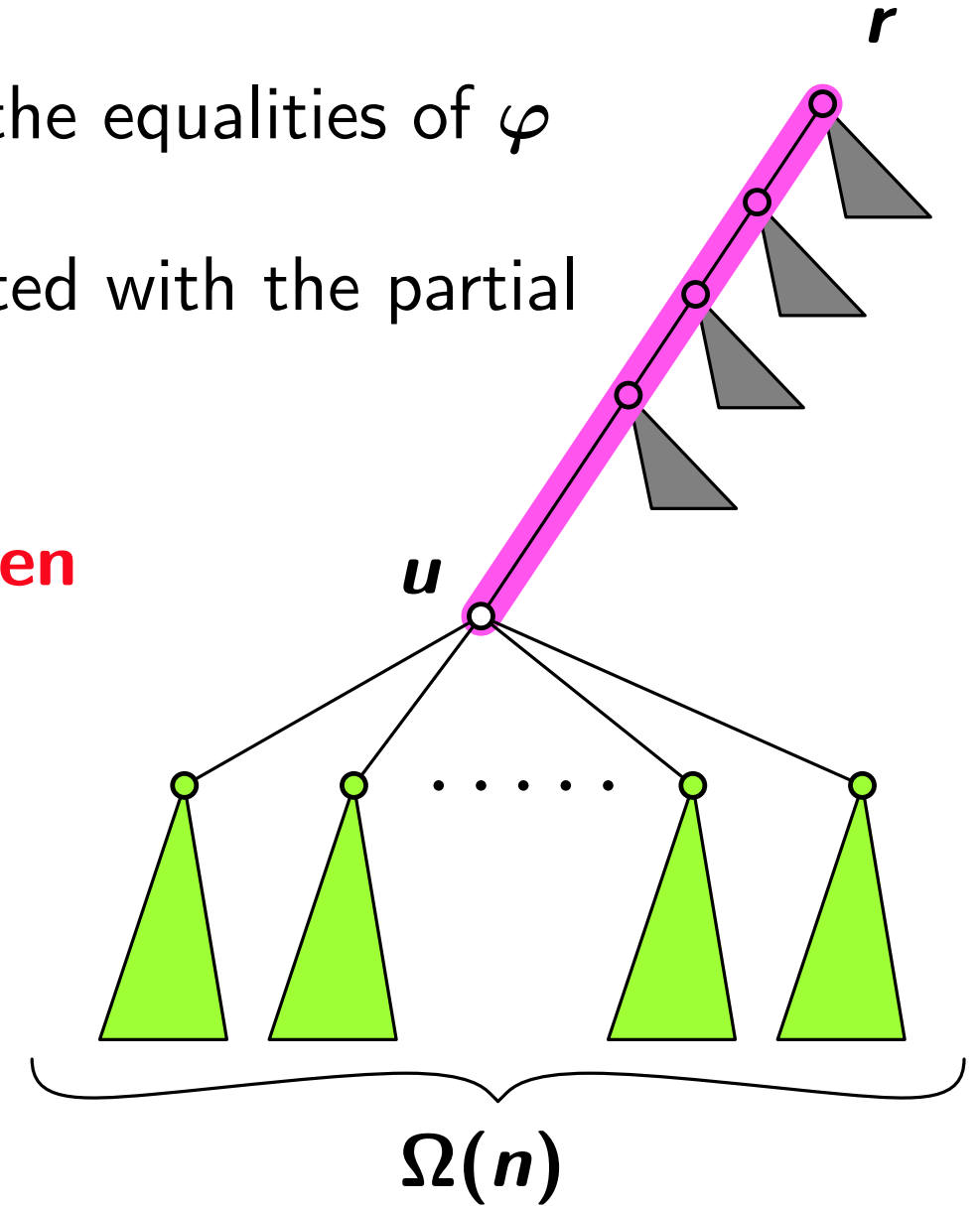
Problems and solutions I

Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

Problem: a node of T can have $\Omega(n)$ children

- Odd DS: we can't keep track of which vertices in **tail** are oddly dominated



Problems and solutions I

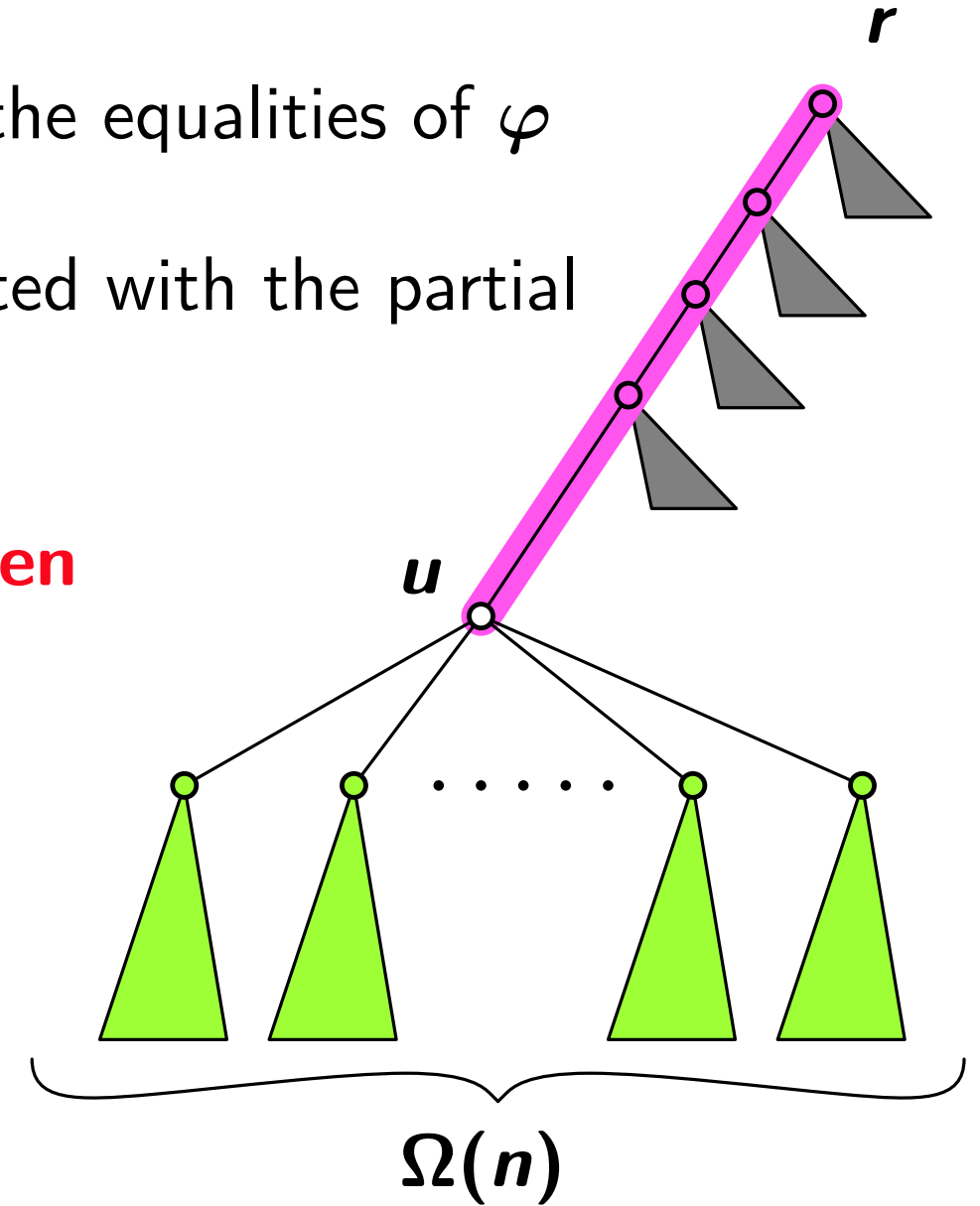
Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

Problem: a node of T can have $\Omega(n)$ children

- Odd DS: we can't keep track of which vertices in **tail** are oddly dominated

→ Solution: Discrete Fourier transform (black magic)



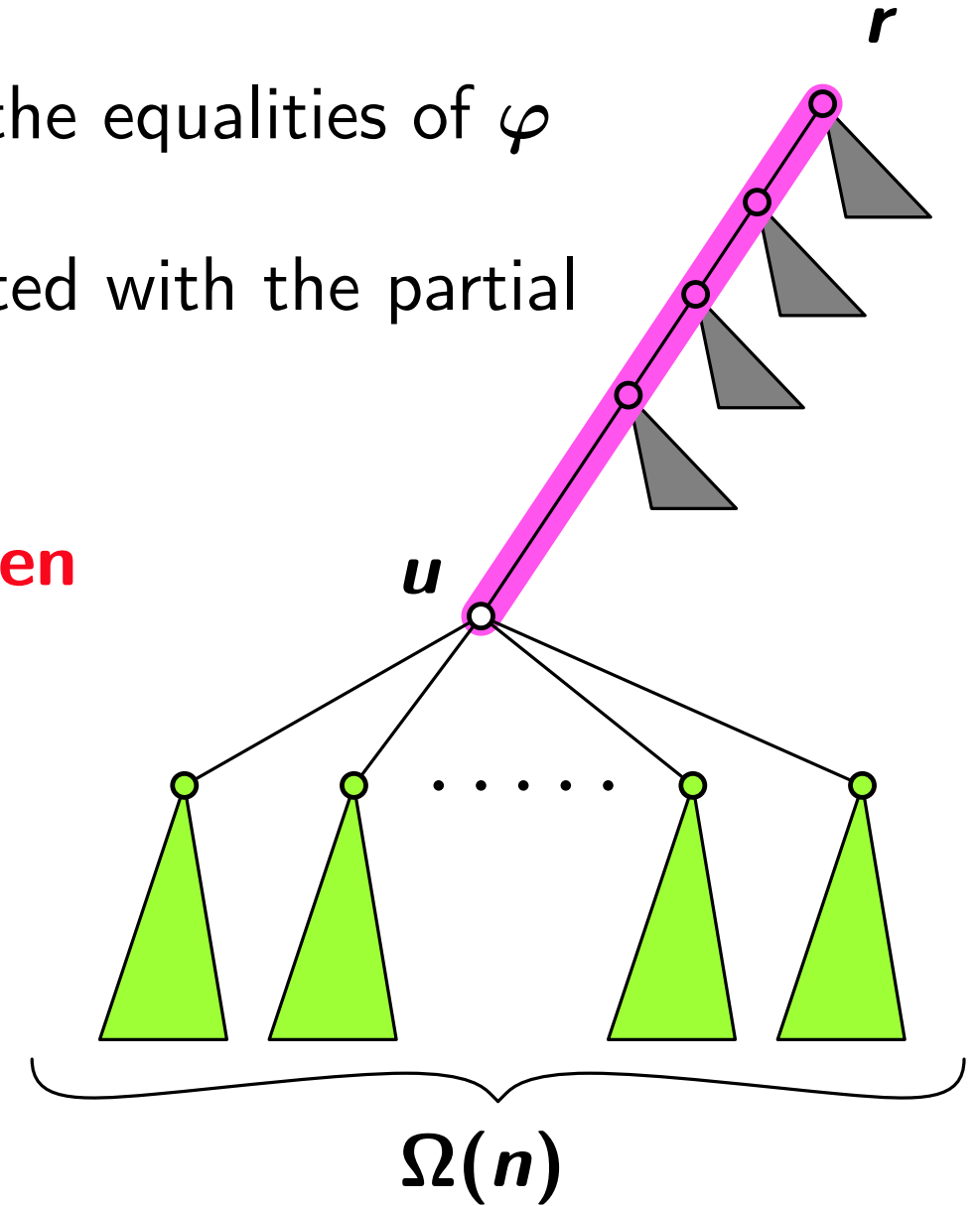
Problems and solutions I

Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

Problem: a node of T can have $\Omega(n)$ children

- We cannot keep track of which equalities of φ are **falsified**



Problems and solutions I

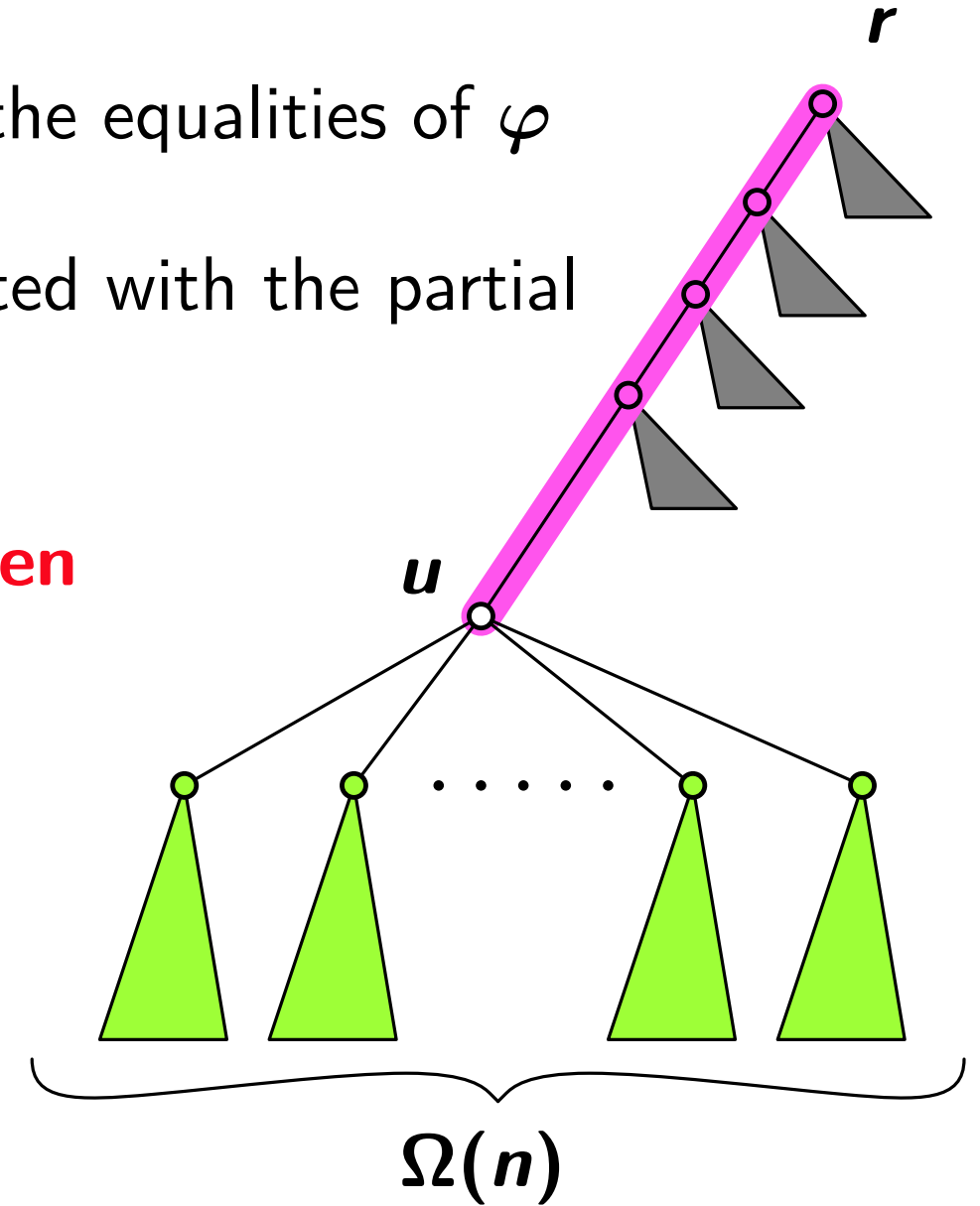
Each index keeps some information about the **tail** and the equalities of φ

- Each entry of a table $Q_u[I]$ is a polynomial associated with the partial solutions **compatible** with the index I

Problem: a node of T can have $\Omega(n)$ children

- We cannot keep track of which equalities of φ are **falsified**

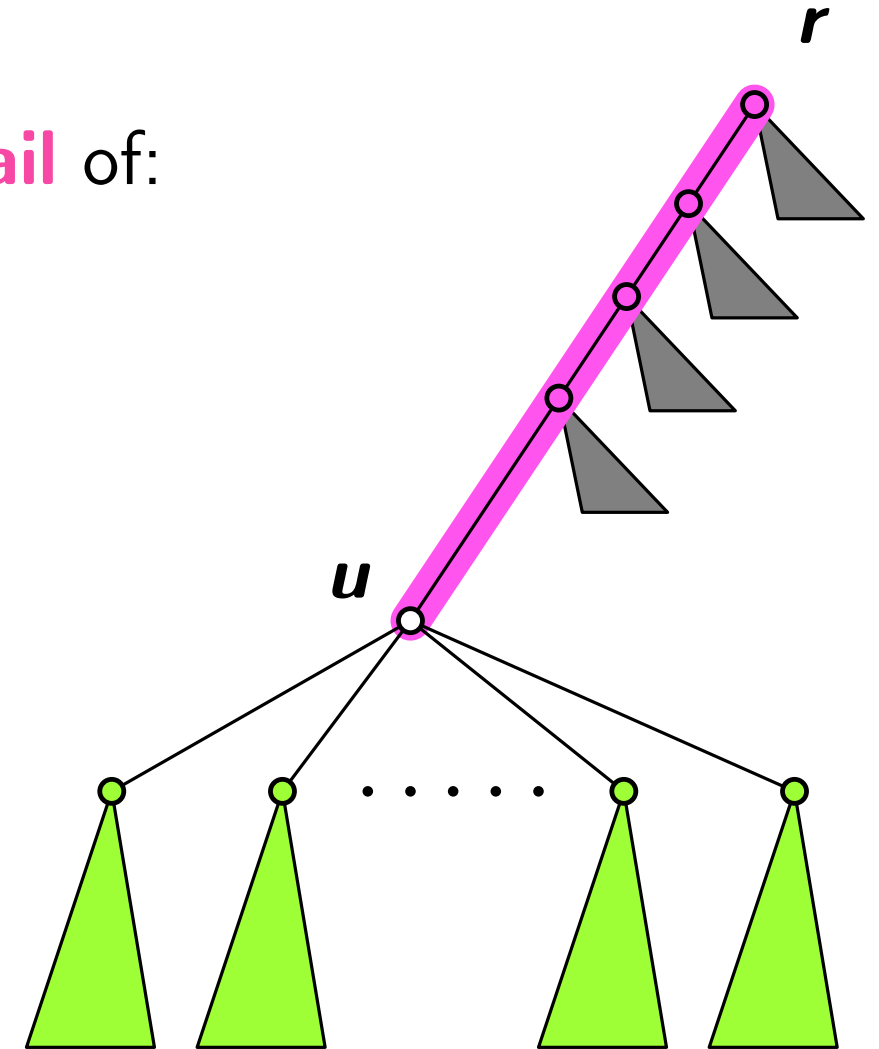
→ Solution: keep track of which equalities are allowed to be false + inclusion-exclusion at the root



Problems and solutions II

To handle $N^\sigma(X, Y)$ an index keep track for each $v \in \text{tail}$ of:

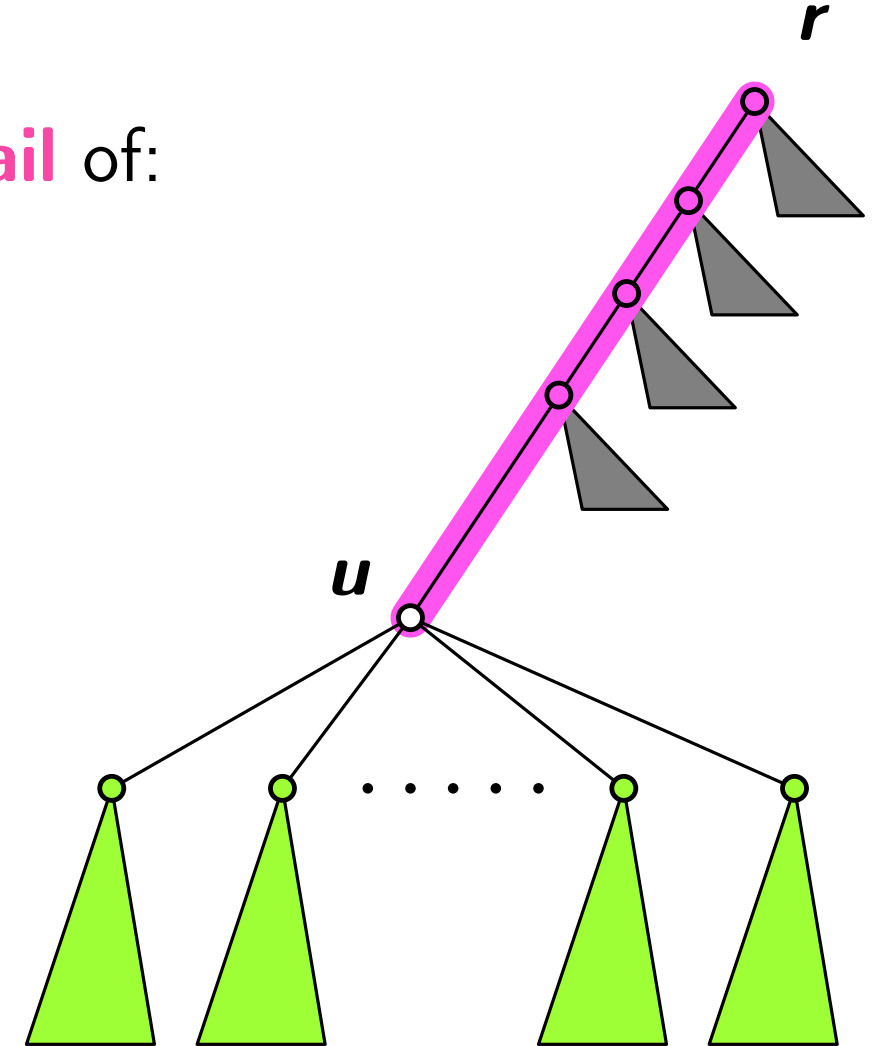
- Number of **expected neighbors** of v up to d_φ
 $\mathbb{E}(v, (X, Y)) \in \{0, \dots, d - 1, \geq d_\varphi\}$



Problems and solutions II

To handle $N^\sigma(X, Y)$ an index keep track for each $v \in \text{tail}$ of:

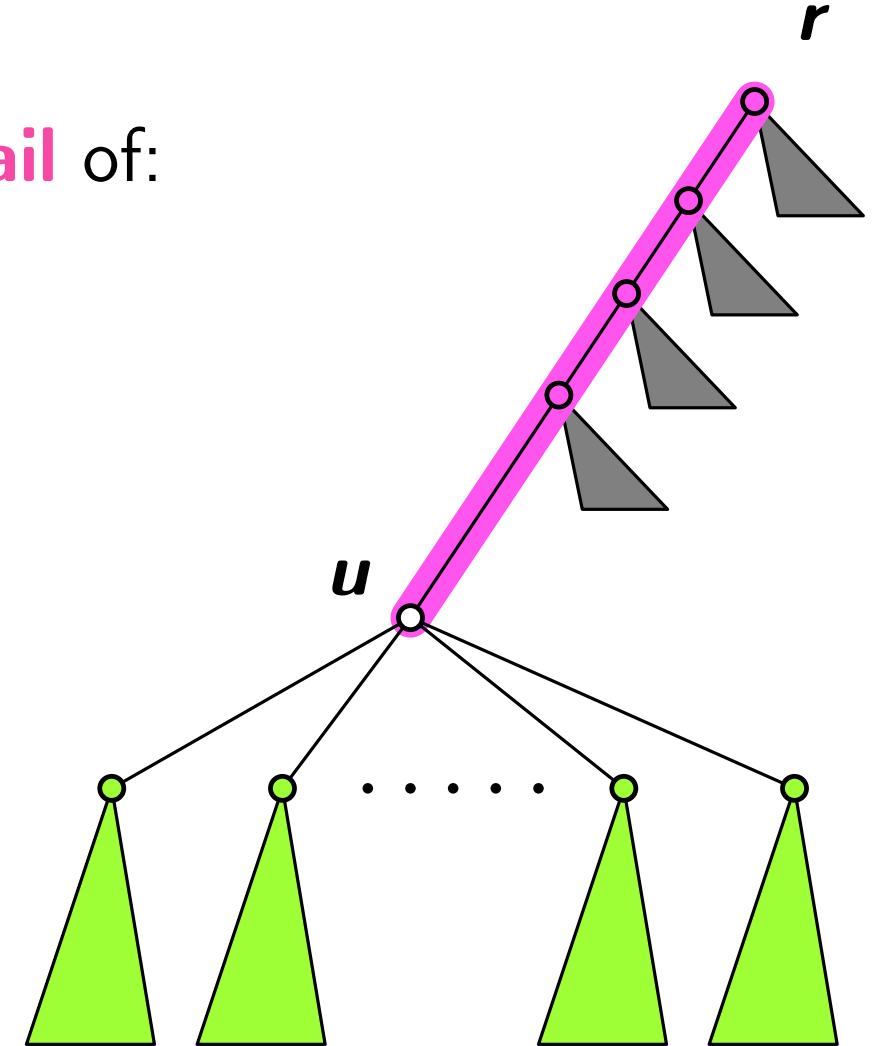
- Number of **expected neighbors** of v up to d_φ
 $\mathbb{E}(v, (X, Y)) \in \{0, \dots, d - 1, \geq d_\varphi\}$
- Number of **neighbors modulo** p_φ
 $\text{pdeg}(v, (X, Y)) \in \{0, \dots, p_\varphi - 1\}$



Problems and solutions II

To handle $N^\sigma(X, Y)$ an index keep track for each $v \in \text{tail}$ of:

- Number of **expected neighbors** of v up to d_φ
 $\mathbb{E}(v, (X, Y)) \in \{0, \dots, d - 1, \geq d_\varphi\}$
- Number of **neighbors modulo** p_φ
 $\text{pdeg}(v, (X, Y)) \in \{0, \dots, p_\varphi - 1\}$



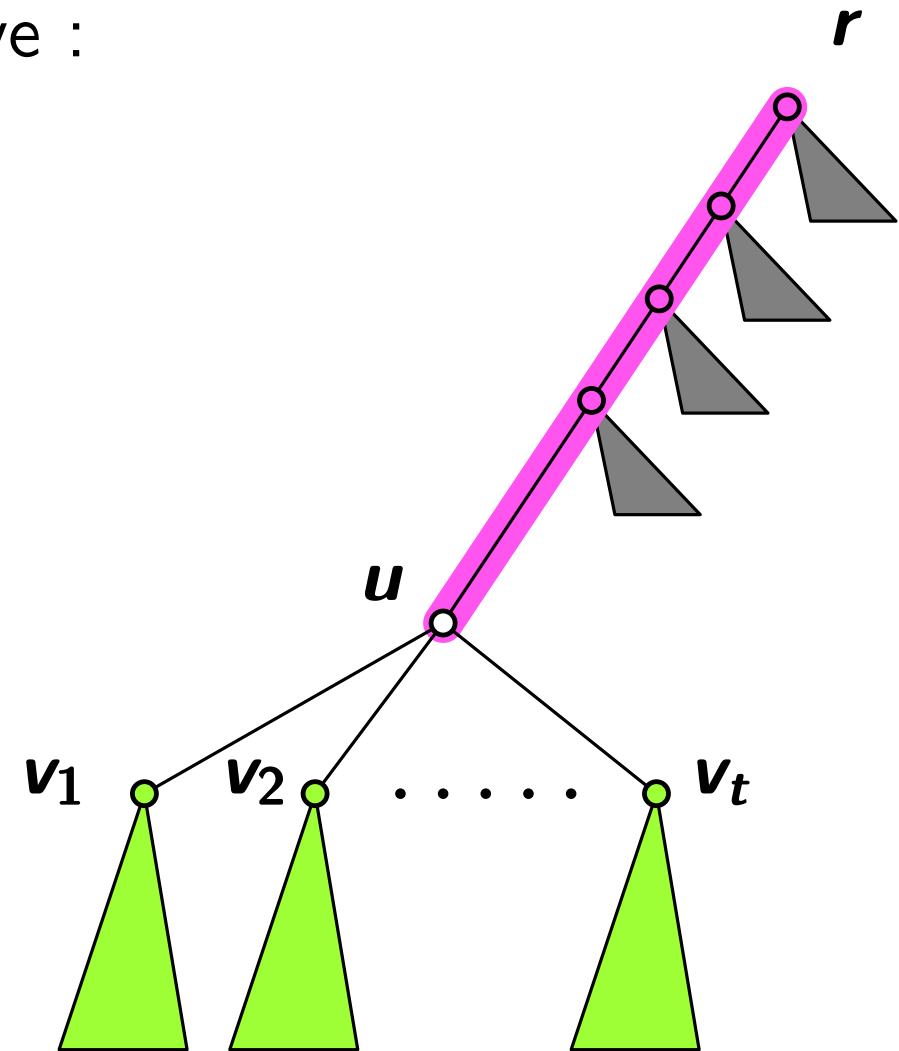
$\mathbb{E}$ and **pdeg** allows to interpret the vertex terms of φ on **tail**

We filter out the partial solution not consistent with $\mathbb{E}$ with incl.-excl. and a counting trick

Join node

For every node u with children $v_1, \dots, v_t$, we have :

$$Q_u[I] \equiv \prod_{i=1}^t Q_{v_i}(I) \pmod{p}$$



Other ingredients

- To handle the connectivity and acyclicity constraints:
→ ♡ **Cut&Count** ♡ [Cygan et al., TALG 2022]
 $n^{\mathcal{O}(q)}$ in running time with q the nb. of acyclicity constraints (removable)

Other ingredients

- To handle the connectivity and acyclicity constraints:
→ ♡ **Cut&Count** ♡ [Cygan et al., TALG 2022]
 $n^{\mathcal{O}(q)}$ in running time with q the nb. of acyclicity constraints (removable)
- To handle the edge sets and expectations $\mathbb{E}$:
→ we generalize a trick used for Hamiltonian Cycle
[Nederlof, Pilipczuk, Swennenhuis, Wegrzycki, DM 2023]

Other ingredients

- To handle the connectivity and acyclicity constraints:
→ ♡ **Cut&Count** ♡ [Cygan et al., TALG 2022]
 $n^{\mathcal{O}(q)}$ in running time with q the nb. of acyclicity constraints (removable)
- To handle the edge sets and expectations $\mathbb{E}$:
→ we generalize a trick used for Hamiltonian Cycle
[Nederlof, Pilipczuk, Swennenhuis, Wegrzycki, DM 2023]
- To reduce the space complexity to $(td + \log n)^{\mathcal{O}(1)}$ for $\mathbf{NEO}_2[\mathbf{FRec}] + \mathbf{K}$:
→ we use a trick based on the Chinese remainder theorem from
[Pilipczuk, Wrochna, TOCT 2018]

Conclusion and open problems

NEO logics can unify and generalize the **best algorithmic applications** of several **width parameters** $\rightarrow$ tree-width, clique-width, rank-width, mim-width, tree-depth,...

Conclusion and open problems

NEO logics can unify and generalize the **best algorithmic applications** of several **width parameters** $\rightarrow$ tree-width, clique-width, rank-width, mim-width, tree-depth,...

- Can we do the same thing for problems solvable in **slightly exp. FPT time**?
 $\rightarrow$ Example: $tw^{\mathcal{O}(tw)} n^{\mathcal{O}(1)}$ time algos for SFVS, SOCT
(Ongoing/neverending project with Lars Jaffke)

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

- Can we derandomized/reduce the space complexity with **NEO₂[FRec]+ACK**?

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

- Can we derandomized/reduce the space complexity with **NEO₂[FRec]+ACK**?
- Can we obtain more time and space efficient FPT algorithms for more general width parameters?
 - Doable in time $2^{\mathcal{O}(d \cdot \ell)} \cdot n^{\mathcal{O}(1)}$ for **shrub-depth** [BCGKMOPvL, TOCT 2024]
 - What about **2-tree-depth**? [Jędrzej Hodor]

[B., Checkan, Stamoulis, SODA 2026] (simplified)

All problems expressible in **NEO₂[FRec]+ACK** can be decided in **time** $2^{\mathcal{O}(\text{td})} n^{\mathcal{O}(1)}$ and $n^{\mathcal{O}(1)}$ **space** given an elimination tree of depth td (Monte-Carlo algorithm)

For **NEO₂[FRec]+K** we can reduce the space complexity to $(\text{td} + \log n)^{\mathcal{O}(1)}$ (determinist algorithm)

- Can we derandomized/reduce the space complexity with **NEO₂[FRec]+ACK**?
- Can we obtain more time and space efficient FPT algorithms for more general width parameters?
 - Doable in time $2^{\mathcal{O}(d \cdot \ell)} \cdot n^{\mathcal{O}(1)}$ for **shrub-depth** [BCGKMOPvL, TOCT 2024]
 - What about **2-tree-depth**? [Jędrzej Hodor]
- What about subexponential FPT space?

Thank you !



ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{x} \xi \wedge \forall v \psi$

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ existentially quantified **vertex set and edge sets variables**

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ existentially quantified **vertex set and edge sets variables**
- ξ **arithmetic formula** over the cardinalities of fixed and quantified sets

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ existentially quantified **vertex set and edge sets variables**
- ξ **arithmetic formula** over the cardinalities of fixed and quantified sets
- $\forall v$ initialize the active vertex

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ **existentially** quantified **vertex set and edge sets variables**
- ξ **arithmetic formula** over the cardinalities of fixed and quantified sets
- $\forall v$ initialize the active vertex
- ψ quantifier-free formula with the operators $\Box^\sigma$ and $\Diamond^\sigma$

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ **existentially** quantified **vertex set and edge sets variables**
- ξ **arithmetic formula** over the cardinalities of fixed and quantified sets
- $\forall v$ initialize the active vertex
- ψ quantifier-free formula with the operators $\Box^\sigma$ and $\Diamond^\sigma$
 - σ is a finitely recognizable (periodic after some constant)

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ **existentially** quantified **vertex set and edge sets variables**
- ξ **arithmetic formula** over the cardinalities of fixed and quantified sets
- $\forall v$ initialize the active vertex
- ψ quantifier-free formula with the operators $\Box^\sigma$ and $\Diamond^\sigma$
 - σ is a finitely recognizable (periodic after some constant)
 - $\Box^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ

ECML

Interpretations use an **active vertex** that change with the operators $\Box^\sigma$ and $\Diamond^\sigma$

An ECML formula is of the form $\exists \bar{X} \xi \wedge \forall v \psi$

- $\exists \bar{X}$ **existentially** quantified **vertex set and edge sets variables**
- ξ **arithmetic formula** over the cardinalities of fixed and quantified sets
- $\forall v$ initialize the active vertex
- ψ quantifier-free formula with the operators $\Box^\sigma$ and $\Diamond^\sigma$
 - σ is a finitely recognizable (periodic after some constant)
 - $\Box^\sigma \varphi$ means the nb. of neighbors of the active vertex satisfying φ is in σ
 - $\Diamond^\sigma \varphi$ means $\neg \Box^\sigma \neg \varphi$